

Labsheet-1

1. Display name, age, and college name

```
public class DisplayInfo {  
    public static void main(String[] args) {  
        String name = "John Doe";  
        int age = 20;  
        String college = "ABC College";  
        System.out.println("Name: " + name);  
        System.out.println("Age: " + age);  
        System.out.println("College: " + college);  
    }  
}
```

2. Arithmetic operations

```
public class ArithmeticOperations {  
    public static void main(String[] args) {  
        int a = 15, b = 4;  
        System.out.println("Addition: " + (a + b));  
        System.out.println("Subtraction: " + (a - b));  
        System.out.println("Multiplication: " + (a * b));  
        System.out.println("Division: " + (a / b));  
        System.out.println("Modulus: " + (a % b));  
    }  
}
```

3. Even or Odd

```
public class EvenOddCheck {  
    public static void main(String[] args) {  
        int num = 7;  
        if (num % 2 == 0)
```

```
System.out.println(num + " is Even");
else
System.out.println(num + " is Odd");
}
}
```

4. Relational operators

```
public class RelationalOperatorsDemo {
public static void main(String[] args) {
int x = 10, y = 20;
System.out.println("x == y: " + (x == y));
System.out.println("x != y: " + (x != y));
System.out.println("x > y: " + (x > y));
System.out.println("x < y: " + (x < y));
System.out.println("x >= y: " + (x >= y));
System.out.println("x <= y: " + (x <= y));
}
}
```

5. Float to int conversion

```
public class FloatToIntConversion {
public static void main(String[] args) {
float num = 12.75f;
int converted = (int) num;
System.out.println("Original float: " + num);
System.out.println("Converted integer: " + converted);
}
}
```

6. Character ASCII value

```
public class CharToAscii {
public static void main(String[] args) {
```

```
char ch = 'A';
int ascii = (int) ch;
System.out.println("Character: " + ch);
System.out.println("ASCII Value: " + ascii);
}
}
```

7. Divisible by both 3 and 5

```
public class DivisibleByThreeAndFive {
    public static void main(String[] args) {
        int num = 30;
        if (num % 3 == 0 && num % 5 == 0)
            System.out.println(num + " is divisible by both 3
            and 5");
        else
            System.out.println(num + " is not divisible by
            both 3 and 5");
    }
}
```

8. Vowel or Consonant

```
public class VowelOrConsonant {
    public static void main(String[] args) {
        char ch = 'e';
        if (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o'
            || ch == 'u' ||
            ch == 'A' || ch == 'E' || ch == 'I' || ch == 'O'
            || ch == 'U') {
            System.out.println(ch + " is a Vowel");
        } else {
            System.out.println(ch + " is a Consonant");
        }
    }
}
```

```
}
```

```
}
```

```
}
```

9. Marks, Total, Percentage, Pass/Fail

```
public class MarksCalculation {  
    public static void main(String[] args) {  
        int m1 = 45, m2 = 50, m3 = 35;  
        int total = m1 + m2 + m3;  
        float percentage = (total / 3.0f);  
        System.out.println("Total Marks: " + total);  
        System.out.println("Percentage: " + percentage);  
        if (m1 >= 40 && m2 >= 40 && m3 >= 40 && percentage >=  
40)  
            System.out.println("Result: Pass");  
        else  
            System.out.println("Result: Fail");  
    }  
}
```

10. Greatest of two numbers

```
public class GreatestOfTwo {  
    public static void main(String[] args) {  
        int a = 25, b = 18;  
        if (a > b)  
            System.out.println(a + " is greater");  
        else  
            System.out.println(b + " is greater");  
    }  
}
```

11. Largest of three numbers (nested if-else)

```

public class LargestOfThree {
    public static void main(String[] args) {
        int a = 25, b = 42, c = 30;
        if (a > b) {
            if (a > c)
                System.out.println(a + " is the largest");
            else
                System.out.println(c + " is the largest");
        } else {
            if (b > c)
                System.out.println(b + " is the largest");
            else
                System.out.println(c + " is the largest");
        }
    }
}

```

12. Leap year check

```

public class LeapYearCheck {
    public static void main(String[] args) {
        int year = 2024;
        if ((year % 4 == 0 && year % 100 != 0) || (year % 400
            == 0))
            System.out.println(year + " is a Leap Year");
        else
            System.out.println(year + " is Not a Leap Year");
    }
}

```

13. Positive, Negative, or Zero

```

public class NumberSignCheck {

```

```

public static void main(String[] args) {
    int num = -10;
    if (num > 0)
        System.out.println(num + " is Positive");
    else if (num < 0)
        System.out.println(num + " is Negative");
    else
        System.out.println("Number is Zero");
}
}

```

14. Character classification

```

public class CharacterClassification {
    public static void main(String[] args) {
        char ch = '#';
        if (ch >= '0' && ch <= '9')
            System.out.println(ch + " is a Digit");
        else if (ch >= 'A' && ch <= 'Z')
            System.out.println(ch + " is an Uppercase
Letter");
        else if (ch >= 'a' && ch <= 'z')
            System.out.println(ch + " is a Lowercase Letter");
        else
            System.out.println(ch + " is a Special
Character");
    }
}

```

15. Armstrong number (3-digit)

```

public class ArmstrongCheck {
    public static void main(String[] args) {

```

```
int num = 153; // 1^3 + 5^3 + 3^3 = 153
int original = num;
int sum = 0;
int d1 = num % 10;
num = num / 10;
int d2 = num % 10;
num = num / 10;
int d3 = num % 10;
sum = (d1 * d1 * d1) + (d2 * d2 * d2) + (d3 * d3 *
d3);
```

```
if (sum == original)
System.out.println(original + " is an Armstrong
Number");
else
System.out.println(original + " is Not an
Armstrong Number");
}
}
```

16. Income tax calculation

```
public class IncomeTaxCalculator {
public static void main(String[] args) {
double income = 750000;
double tax = 0;
if (income <= 250000)
tax = 0;
else if (income <= 500000)
tax = (income - 250000) * 0.05;
else if (income <= 1000000)
tax = (250000 * 0.05) + (income - 500000) * 0.20;
```

```

else
tax = (250000 * 0.05) + (500000 * 0.20) + (income
- 1000000) * 0.30;
System.out.println("Income: " + income);
System.out.println("Calculated Tax: " + tax);
}
}

```

17. Palindrome number

```

public class PalindromeCheck {
public static void main(String[] args) {
int num = 121;
int original = num;
int reversed = 0;
int d1 = num % 10;
num = num / 10;
int d2 = num % 10;
num = num / 10;
int d3 = num % 10;
reversed = (d1 * 100) + (d2 * 10) + d3;
if (original == reversed)
System.out.println(original + " is a Palindrome");
else
System.out.println(original + " is Not a
Palindrome");
}
}

```

18. Prime number check

```

public class PrimeCheck {
public static void main(String[] args) {

```



```

int num = 17;
boolean isPrime = true;
if (num <= 1) {
    isPrime = false;
} else {
    for (int i = 2; i <= num / 2; i++) {
        if (num % i == 0) {

            isPrime = false;
            break;
        }
    }
}
if (isPrime)
    System.out.println(num + " is a Prime Number");
else
    System.out.println(num + " is Not a Prime
    Number");
}
}

```

19. Operator validation and calculation

```

public class OperatorCalculator {
    public static void main(String[] args) {
        int a = 20, b = 10;
        char op = '*';
        if (op == '+')
            System.out.println("Result: " + (a + b));
        else if (op == '-')
            System.out.println("Result: " + (a - b));
        else if (op == '*')

```

```
System.out.println("Result: " + (a * b));
else if (op == '/')
System.out.println("Result: " + (a / b));
else
System.out.println("Invalid Operator");
}
}
```

20. Marriage eligibility

```
public class MarriageEligibility {
public static void main(String[] args) {
int age = 19;
char gender = 'F'; // 'M' for male, 'F' for female
if (gender == 'M') {
if (age >= 21)
System.out.println("Male, Age " + age + ":
Eligible for Marriage");
else
System.out.println("Male, Age " + age + ": Not
Eligible for Marriage");
} else if (gender == 'F') {
if (age >= 18)
System.out.println("Female, Age " + age + ":
Eligible for Marriage");
else
System.out.println("Female, Age " + age + ":
Not Eligible for Marriage");
} else {
System.out.println("Invalid Gender");
}
```

```
}  
}
```

21. Valid Triangle(sum of angles=180°)

```
public class ValidTriangleCheck {  
    public static void main(String[] args) {  
        int angle1 = 60, angle2 = 60, angle3 = 60;  
        if (angle1 + angle2 + angle3 == 180)  
            System.out.println("Valid Triangle");  
        else  
            System.out.println("Not a Valid Triangle");  
    }  
}
```

22.

Square or Rectangle

```
public class SquareOrRectangle {  
    public static void main(String[] args) {  
        int length = 20, breadth = 20;  
        if (length == breadth)  
            System.out.println("It is a Square");  
        else  
            System.out.println("It is a Rectangle");  
    }  
}
```

23.

Celsius to

Fahrenheit Conversion

```
public class CelsiusToFahrenheit {  
    public static void main(String[] args) {  
        float celsius = 37.0f;  
        float fahrenheit = (celsius * 9 / 5) + 32;
```

```
System.out.println("Celsius: " + celsius);
System.out.println("Fahrenheit: " + fahrenheit);
}
}
```

24.

Perfect Number Check

```
public class PerfectNumberCheck {
    public static void main(String[] args) {
        int num = 28;
        int sum = 0;
        for (int i = 1; i < num; i++) {
            if (num % i == 0) {
                sum += i;
            }
        }
        if (sum == num)
            System.out.println(num + " is a Perfect
            Number");
        else
            System.out.println(num + " is Not a Perfect
            Number");
    }
}
```

25.

Logical Operator Demo

```
public class LogicalOperatorDemo {
    public static void main(String[] args) {
        boolean a = true, b = false;
```

```

char op = '&'; // '&' for AND, '|' for OR
if (op == '&')
    System.out.println("Result: " + (a && b));
else if (op == '|')
    System.out.println("Result: " + (a || b));
else
    System.out.println("Invalid Logical
Operator");
}
}

```

26.

First and

Last Digit

Check (5-digit number)

```

public class FirstLastDigitCheck {
    public static void main(String[] args) {
        int num = 12341;
        int lastDigit = num % 10;
        int firstDigit = num / 10000;
        if (firstDigit == lastDigit)
            System.out.println(num + ": First and Last digits
are the same");
        else
            System.out.println(num + ": First and Last digits
are different");

    }
}

```

27. Profit or Loss

```

public class ProfitOrLoss {

```

```

public static void main(String[] args) {
    int costPrice = 500, sellingPrice = 650;
    if (sellingPrice > costPrice)
        System.out.println("Profit: " + (sellingPrice -
            costPrice));
    else if (sellingPrice < costPrice)
        System.out.println("Loss: " + (costPrice -
            sellingPrice));
    else
        System.out.println("No Profit, No Loss");
}
}

```

28. Number of Digits

```

public class DigitCount {
    public static void main(String[] args) {
        int num = 12345;
        int count = 0;
        int temp = num;
        while (temp != 0) {
            count++;
            temp = temp / 10;
        }
        System.out.println("Number: " + num);
        System.out.println("Digits: " + count);
    }
}

```

29. Height Classification

```

public class HeightClassification {
    public static void main(String[] args) {

```

```
int height = 165;
if (height < 150)
    System.out.println("Short");
else if (height <= 170)
    System.out.println("Average");
else
    System.out.println("Tall");
}
}
```

30. Simple Calculator

```
public class SimpleCalculator {
    public static void main(String[] args) {
        int a = 20, b = 5;
        char op = '/';
        if (op == '+')
            System.out.println("Result: " + (a + b));
        else if (op == '-')
            System.out.println("Result: " + (a - b));
        else if (op == '*')
            System.out.println("Result: " + (a * b));
        else if (op == '/')
            System.out.println("Result: " + (a / b));
        else
            System.out.println("Invalid Operator");
    }
}
```

Labsheet-2

Q1. Area of a Triangle using Heron's Formula

```
class TriangleArea {  
    public static void main(String[] args) {  
        double a = 5, b = 6, c = 7; // sides of triangle  
        double s = (a + b + c) / 2; // semi-perimeter  
        double area = Math.sqrt(s * (s - a) * (s - b) * (s -  
c));  
        System.out.println("Area of Triangle: " + area);  
    }  
}
```

Q2. Compound Interest

```
class CompoundInterest {  
    public static void main(String[] args) {  
        double principal = 10000, rate = 5, time = 2;  
        double amount = principal * Math.pow((1 + rate / 100),  
time);  
        double ci = amount - principal;  
        System.out.println("Compound Interest: " + ci);  
    }  
}
```

Q3. Distance between two points

```
class Distance2D {  
    public static void main(String[] args) {  
        double x1 = 2, y1 = 3, x2 = 7, y2 = 8;  
        double distance = Math.sqrt(Math.pow(x2 - x1, 2) +  
Math.pow(y2 - y1, 2));  
        System.out.println("Distance between points: " +
```



```
distance);
```

```
}
```

```
}
```

Q4. Prefix & Postfix Increment (Visitor Counter)

```
class VisitorCounter {
```

```
public static void main(String[] args) {
```

```
int visitors = 0;
```

```
System.out.println("Visitors entering store:");
```

```
System.out.println("Prefix: " + (++visitors)); // 1
```

```
System.out.println("Postfix: " + (visitors++)); // 1
```

```
but increments after
```

```
System.out.println("Total Visitors: " + visitors); //
```

```
2
```

```
}
```

```
}
```

Q5. Negative Value using Unary Operator

```
class NegativeValue {
```

```
public static void main(String[] args) {
```

```
int num = 15;
```

```
int negative = -num;
```

```
System.out.println("Original: " + num + ", Negative: "
```

```
+ negative);
```

```
}
```

```
}
```

Q6. Reduce number by half

```
class ReduceByHalf {
```

```
public static void main(String[] args) {
```

```
double num = 100;
```

```
int steps = 0;
```

```

while (num >= 1) {
    num /= 2;
    steps++;
}

System.out.println("Steps to reduce below 1: " +
steps);
}
}

```

Q7. Accumulate Rainfall Data

```

class RainfallTotal {
    public static void main(String[] args) {
        int[] rainfall = {12, 15, 10, 8, 20, 5, 14};
        int total = 0;
        for (int i = 0; i < rainfall.length; i++) {
            total += rainfall[i];
        }

        System.out.println("Total Rainfall in 7 days: " +
total);
    }
}

```

Q8. Valid Triangle by Angles

```

class ValidTriangle {
    public static void main(String[] args) {
        int a = 60, b = 60, c = 60;
        if ((a + b + c == 180) && a > 0 && b > 0 && c > 0)
            System.out.println("Valid Triangle");
        else
            System.out.println("Invalid Triangle");
    }
}

```

Q9. Lexicographic Order (without built-in)

```
class StringCompare {
public static void main(String[] args) {
String s1 = "apple", s2 = "apricot";
int minLen = Math.min(s1.length(), s2.length());
boolean equal = true;
for (int i = 0; i < minLen; i++) {
if (s1.charAt(i) < s2.charAt(i)) {
System.out.println(s1 + " comes before " +
s2);
equal = false;
break;
} else if (s1.charAt(i) > s2.charAt(i)) {
System.out.println(s2 + " comes before " +
s1);
equal = false;
break;
}
}
if (equal) {
if (s1.length() < s2.length())
System.out.println(s1 + " comes before " +
s2);
else if (s1.length() > s2.length())
System.out.println(s2 + " comes before " +
s1);
else
System.out.println("Both strings are equal");
}
```

```
}  
}
```

Logical Operators (Q10–Q11)

Q10. Leap Year & Within Range

```
class LeapYearRange {  
    public static void main(String[] args) {  
        int year = 2024;  
        int start = 2000, end = 2030;  
        boolean isLeap = (year % 400 == 0) || (year % 4 == 0  
        && year % 100 != 0);  
        boolean inRange = (year >= start && year <= end);  
        if (isLeap && inRange)  
            System.out.println(year + " is a Leap Year and  
            within range.");  
        else  
            System.out.println(year + " does not satisfy both  
            conditions.");  
    }  
}
```

Q11. Student Pass Condition

```
class StudentPass {  
    public static void main(String[] args) {  
        int theory = 45;  
        int practical = 40;  
        int total = (theory + practical) / 2;  
        if ((theory >= 40 && practical >= 50) || total >= 50)  
            System.out.println("Student Passed");  
        else  
            System.out.println("Student Failed");  
    }  
}
```

```
}  
}
```

Ternary Operator (Q12–Q13)

Q12. Smallest of Four Numbers

```
class SmallestOfFour {  
    public static void main(String[] args) {  
        int a = 10, b = 5, c = 20, d = 2;  
        int smallest = (a < b ? (a < c ? (a < d ? a : d) : (c  
        < d ? c : d))  
        : (b < c ? (b < d ? b : d) : (c  
        < d ? c : d)));  
        System.out.println("Smallest Number: " + smallest);  
    }  
}
```

Q13. Character Check

```
class CharacterCheck {  
    public static void main(String[] args) {  
        char ch = '#';  
        String result =  
        (ch >= 'a' && ch <= 'z' || ch >= 'A' && ch <= 'Z')  
        ?  
        ((ch=='a'||ch=='e'||ch=='i'||ch=='o'||ch=='u'||  
        ch=='A'||ch=='E'||ch=='I'||ch=='O'||ch=='U'  
        )  
        ? "Vowel" : "Consonant")  
        : (ch >= '0' && ch <= '9') ? "Digit" : "Special  
        Symbol";  
        System.out.println("Character is: " + result);  
    }  
}
```

```
}
```

Bitwise Operators (Q14–Q15)

Q14. Swap Using XOR

```
class SwapXOR {  
    public static void main(String[] args) {  
        int x = 10, y = 20;  
        System.out.println("Before Swap: x=" + x + ", y=" +  
y);  
        x = x ^ y;  
        y = x ^ y;  
        x = x ^ y;  
        System.out.println("After Swap: x=" + x + ", y=" + y);  
    }  
}
```

Q15. Count Set Bits

```
class CountSetBits {  
    public static void main(String[] args) {  
        int num = 29; // 11101 in binary  
        int count = 0;  
        while (num > 0) {  
            count += (num & 1);  
            num >>= 1;  
        }  
        System.out.println("Number of set bits: " + count);  
    }  
}
```

Shift Operators (Q16–Q17)

Q16. Fast Multiplication & Division by 2

```

class ShiftMultiplyDivide {
public static void main(String[] args) {
int num = 8;
System.out.println("Original: " + num);
System.out.println("Multiply by 2: " + (num << 1));
System.out.println("Divide by 2: " + (num >> 1));
}
}

```

Q17. Cyclic Left Rotate by 2

```

class CyclicRotate {
public static void main(String[] args) {
int num = 19; // binary 10011
int bits = 32; // assuming int is 32-bit
int rotated = (num << 2) | (num >>> (bits - 2));
System.out.println("After Left Rotate by 2: " +
rotated);
}
}

```

Q18. Fibonacci (first 20 terms using for loop)

```

class FibonacciForLoop {
public static void main(String[] args) {
int first = 0, second = 1;
System.out.print("First 20 Fibonacci terms: ");
for (int i = 1; i <= 20; i++) {
System.out.print(first + " ");
int next = first + second;
first = second;
second = next;
}
}

```

```
}
```

```
}
```

Q19. Armstrong numbers (1–1000 using for loop)

```
class ArmstrongForLoop {  
    public static void main(String[] args) {  
        System.out.println("Armstrong numbers between 1 and  
        1000:");  
        for (int num = 1; num <= 1000; num++) {  
            int sum = 0, temp = num;  
            int digits = String.valueOf(num).length();  
            while (temp > 0) {  
                int digit = temp % 10;  
                sum += Math.pow(digit, digits);  
                temp /= 10;  
            }  
            if (sum == num) {  
                System.out.print(num + " ");  
            }  
        }  
    }  
}
```

Q20. Accept password until correct (do-while loop)

```
import java.util.Scanner;  
class PasswordDoWhile {  
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);  
        final String correctPassword = "java123";  
        String enteredPassword;  
        do {
```



```

System.out.print("Enter password: ");
enteredPassword = scanner.nextLine();
} while (!enteredPassword.equals(correctPassword));
System.out.println("Access Granted!");
scanner.close();
}
}

```

Q21. Factors of a number (do-while loop)

```

import java.util.Scanner;
class FactorsDoWhile {
public static void main(String[] args) {
Scanner scanner = new Scanner(System.in);
System.out.print("Enter a number: ");
int number = scanner.nextInt();
int divisor = 1;
System.out.println("Factors of " + number + ":");
do {
if (number % divisor == 0) {
System.out.print(divisor + " ");
}
divisor++;
} while (divisor <= number);
scanner.close();
}
}

```

Q22. Reverse digits (while loop)

```

import java.util.Scanner;
class ReverseWhileLoop {
public static void main(String[] args) {

```

```

Scanner scanner = new Scanner(System.in);
System.out.print("Enter an integer: ");
int number = scanner.nextInt();
int reversed = 0;
while (number != 0) {
    int digit = number % 10;
    reversed = reversed * 10 + digit;
    number /= 10;
}
System.out.println("Reversed number: " + reversed);
scanner.close();
}
}

```

Q23. Palindrome number (while loop)

```

import java.util.Scanner;

class PalindromeWhileLoop {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter a number: ");
        int number = scanner.nextInt();
        int temp = number, reversed = 0;
        while (temp != 0) {
            int digit = temp % 10;
            reversed = reversed * 10 + digit;
            temp /= 10;
        }
        if (number == reversed) {
            System.out.println(number + " is a Palindrome.");
        } else {

```

```
System.out.println(number + " is NOT a  
Palindrome.");  
}  
scanner.close();  
}  
}
```

Q24. Max & Min in array (for-each loop)

```
class ArrayMinMaxForEach {  
    public static void main(String[] args) {  
        int[] numbers = {45, 12, 89, 5, 67, 23};  
        int max = numbers[0], min = numbers[0];  
        for (int num : numbers) {  
            if (num > max) max = num;  
            if (num < min) min = num;  
        }  
        System.out.println("Maximum: " + max);  
        System.out.println("Minimum: " + min);  
    }  
}
```

Q25. Average marks of students (2D array, for-each loop)

```
class AverageMarksForEach {  
    public static void main(String[] args) {  
        int[][] marks = {  
            {85, 90, 78},  
            {72, 88, 95},  
            {91, 84, 89}  
        };  
        int totalMarks = 0, count = 0;  
        for (int[] student : marks) {
```

```

for (int mark : student) {
totalMarks += mark;
count++;
}
}
double average = (double) totalMarks / count;
System.out.println("Average Marks: " + average);

}
}

```

Q26. Day number → Weekday/Weekend (switch-case)

```

import java.util.Scanner;
class DaySwitchCase {
public static void main(String[] args) {
Scanner scanner = new Scanner(System.in);
System.out.print("Enter day number (1-7): ");
int day = scanner.nextInt();
switch (day) {
case 1: case 2: case 3: case 4: case 5:
System.out.println("It's a Weekday.");
break;
case 6: case 7:
System.out.println("It's a Weekend.");
break;
default:
System.out.println("Invalid day number!");
}
scanner.close();
}
}

```

```
}
```

Q27. Grade → Meaning (switch-case)

```
import java.util.Scanner;

class GradeSwitchCase {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter grade (A-F): ");
        char grade = scanner.next().charAt(0);
        switch (Character.toUpperCase(grade)) {
            case 'A': System.out.println("Excellent"); break;
            case 'B': System.out.println("Good"); break;
            case 'C': System.out.println("Average"); break;

            case 'D': System.out.println("Below Average");
                break;
            case 'E': System.out.println("Poor"); break;
            case 'F': System.out.println("Fail"); break;
            default: System.out.println("Invalid grade!");
        }
        scanner.close();
    }
}
```

Q28. Skip perfect squares (continue)

```
class SkipPerfectSquares {
    public static void main(String[] args) {
        System.out.println("Numbers from 1 to 50 (excluding
        perfect squares):");
        for (int i = 1; i <= 50; i++) {
            int sqrt = (int) Math.sqrt(i);
            if (sqrt * sqrt == i) {
```

```

continue; // skip perfect square
}
System.out.print(i + " ");
}
}
}

```

Q29. Random numbers until divisible by 7 & 13 (break)

```

import java.util.Random;
class RandomBreak {
public static void main(String[] args) {
Random rand = new Random();
while (true) {
int num = rand.nextInt(100) + 1;
System.out.println("Generated: " + num);
if (num % 7 == 0 && num % 13 == 0) {
System.out.println("Found number divisible by
both 7 and 13: " + num);

break;
}
}
}
}

```

Q30. Mixed Advanced

```

import java.util.Scanner;
class MixedAdvanced {
public static void main(String[] args) {
Scanner scanner = new Scanner(System.in);
System.out.print("Enter a number: ");

```

```

int number = scanner.nextInt();

// 1. Check if power of 4 using shift operators
boolean isPowerOf4 = (number > 0) && ((number &
(number - 1)) == 0) && ((number - 1) % 3 == 0);
System.out.println(number + (isPowerOf4 ? " is a power
of 4." : " is NOT a power of 4."));

// 2. Toggle 3rd bit (0-based index)
int toggled = number ^ (1 << 2);
System.out.println("After toggling 3rd bit: " +
toggled);

// 3 & 4. Multiplication table with continue & break
System.out.println("Multiplication Table of " + number
+ ":");
for (int i = 1; i <= 20; i++) {
int product = number * i;
if (product % 6 == 0) continue; // skip multiples
of 6
if (product % 48 == 0) break; // stop at
multiple of 48
System.out.println(number + " x " + i + " = " +
product);
}
scanner.close();
}
}

```

Labsheet-3

```
class Prog3{
    public static void main(String[] args) {

        //Q1
        /*Scanner scanner = new Scanner(System.in);
        int[] numbers = new int[5];

        System.out.println("Enter 5 integers:");
        for (int i = 0; i < numbers.length; i++) {
            numbers[i] = scanner.nextInt();
        }
        System.out.println("You entered:");
        for (int num : numbers) {
            System.out.print(num + " ");
        }*/

        //Q2
        /*Scanner scanner = new Scanner(System.in);
        int[] numbers = new int[10];
        System.out.println("Enter 10 integers:");
        for (int i = 0; i < numbers.length; i++) {
            numbers[i] = scanner.nextInt();
        }
        System.out.println("Numbers in reverse order:");
        for (int i = numbers.length - 1; i >= 0; i--) {
            System.out.print(numbers[i] + " ");
        } */

        //Q3
```



```
/*int[] numbers = { 5, 10, 15, 20, 25 };  
  
int sum = 0;  
for (int num : numbers) {  
    sum += num;  
}  
System.out.println("Sum of elements = " + sum); */
```

//Q4

```
/* int[] numbers = { 45, 12, 89, 33, 76 };  
  
int max = numbers[0];  
for (int num : numbers) {  
    if (num > max) {  
        max = num;  
    }  
}  
System.out.println("Maximum element = " + max); */
```

//Q5

```
/* int[] numbers = { 10, 25, 33, 40, 55 };  
  
int evenCount = 0, oddCount = 0;  
for (int num : numbers) {  
    if (num % 2 == 0) {  
        evenCount++;  
    } else {  
        oddCount++;  
    }  
}  
System.out.println("Even count = " + evenCount);  
System.out.println("Odd count = " + oddCount); */
```

//Q6

```
/*int[] source = { 5, 10, 15, 20, 25 };  
int[] destination = new int[source.length];  
for (int i = 0; i < source.length; i++) {  
    destination[i] = source[i];  
}  
System.out.println("Copied array:");  
for (int num : destination) {  
    System.out.print(num + " ");  
} */
```

//Q7

```
/*Scanner scanner = new Scanner(System.in);  
int[][] matrix = new int[3][3];  
System.out.println("Enter 3x3 matrix:");  
for (int i = 0; i < 3; i++) {  
    for (int j = 0; j < 3; j++) {  
        matrix[i][j] = scanner.nextInt();  
    }  
}  
System.out.println("Matrix is:");  
for (int[] row : matrix){  
    for (int val : row) {  
        System.out.print(val + " ");  
    }  
    System.out.println();  
} */
```

//Q8

```
/*int[][] matrix = { { 1, 2, 3 }, { 4, 5, 6 }, { 7,
```

```

8, 9 } }];

int sum = 0;
for (int[] row : matrix) {
    for (int val : row) {
        sum += val;
    }
}

System.out.println("Sum of elements = " + sum); */

```

```

//Q9
/* int[][] matrix = { { 1, 2, 3 }, { 4, 5, 6 }, { 7,8, 9 } };
for (int i = 0; i < matrix.length; i++) {
    int rowSum = 0;
    for (int j = 0; j < matrix[i].length; j++) {
        rowSum += matrix[i][j];
    }
    System.out.println("Row " + (i + 1) + " sum = " + rowSum);
} */

```

```

//Q10
/*int[][] matrix = { { 1, 2, 3 }, { 4, 5, 6 }, { 7,
8, 9 } }];
for (int j = 0; j < 3; j++) {
    int colSum = 0;
    for (int i = 0; i < 3; i++) {
        colSum += matrix[i][j];
    }
    System.out.println("Column " + (j + 1) + " sum= " + colSum);
} */

```

//Q11

```
/*int[][] matrix = { { 1, 2, 3 }, { 4, 5, 6 }, { 7,
8, 9 } };

int[][] transpose = new int[3][3];

for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        transpose[j][i] = matrix[i][j];
    }
}

System.out.println("Transpose matrix:");

for (int[] row : transpose) {
    for (int val : row) {
        System.out.print(val + " ");
    }
}

System.out.println();
} */
```

//Q12

```
/*int[][] a = { { 1, 2, 3 }, { 4, 5, 6 }, { 7, 8, 9
} };

int[][] b = { { 9, 8, 7 }, { 6, 5, 4 }, { 3, 2, 1
} };

int[][] sum = new int[3][3];

for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        sum[i][j] = a[i][j] + b[i][j];
    }
}

System.out.println("Matrix Addition:");

for (int[] row : sum) {
```

```

    for (int val : row) {
        System.out.print(val + " ");
    }
    System.out.println();
} */

```

//Q13

```

/*int[][] a = { { 1, 2, 3 }, { 4, 5, 6 }, { 7, 8, 9
} };
int[][] b = { { 9, 8, 7 }, { 6, 5, 4 }, { 3, 2, 1
} };
int[][] product = new int[3][3];
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        for (int k = 0; k < 3; k++) {
            product[i][j] += a[i][k] * b[k][j];
        }
    }
}
System.out.println("Matrix Multiplication:");
for (int[] row : product) {
    for (int val : row) {
        System.out.print(val + " ");
    }
    System.out.println();
} */

```

//Q14

```

/*Scanner scanner = new Scanner(System.in);
int[][][] arr = new int[2][2][2];

```

```
System.out.println("Enter elements for 2x2x2 array:");
```

```
for (int i = 0; i < 2; i++) {
```

```
for (int j = 0; j < 2; j++) {
```

```
for (int k = 0; k < 2; k++) {
```

```
arr[i][j][k] = scanner.nextInt();
```

```
}
```

```
}
```

```
}
```

```
System.out.println("Array elements are:");
```

```
for (int i = 0; i < 2; i++) {
```

```
for (int j = 0; j < 2; j++) {
```

```
for (int k = 0; k < 2; k++) {
```

```
System.out.print(arr[i][j][k] + " ");
```

```
}
```

```
System.out.println();
```

```
}
```

```
System.out.println();
```

```
} */
```

```
//Q15
```

```
/*int[][][] arr = { { { 1, 2 }, { 3, 4 } }, { { 5, 6}, { 7, 8 } } };
```

```
int sum = 0;
```

```
for (int[][] twoD : arr) {
```

```
for (int[] oneD : twoD) {
```

```
for (int val : oneD) {
```

```
sum += val;
```

```
}
```

```
}
```

```
}
```

```
System.out.println("Sum of elements = " + sum); */
```

//Q16

```
/* int[][][] arr = { { { 1, -2 }, { -3, 4 } }, { { -5, 6 }, { -7, 8 } } };
```



```
int positiveCount = 0, negativeCount = 0;
```



```
for (int[][] twoD : arr) {
```



```
    for (int[] oneD : twoD) {
```



```
        for (int val : oneD) {
```



```
            if (val >= 0)
```



```
                positiveCount++;
```



```
            else
```



```
                negativeCount++;
```



```
        }
```



```
    }
```



```
}
```



```
System.out.println("Positive count = " +
```



```
positiveCount);
```



```
System.out.println("Negative count = " +
```



```
negativeCount); */
```

//Q17

```
/*int[][][] arr = { { { 1, 2 }, { 3, 4 } }, { { 5, 6
```



```
}, { 7, 8 } } };
```



```
int max = arr[0][0][0];
```



```
for (int[][] twoD : arr) {
```



```
    for (int[] oneD : twoD) {
```



```
        for (int val : oneD) {
```



```
            if (val > max)
```



```
                max = val;
```



```
        }
```



```
    }
```



```
}
```

```

}

System.out.println("Maximum element = " + max); */

//Q18
/* int[][][] arr = { { { 1, 2 }, { 3, 4 } }, { { 5, 6}, { 7, 8 } } };
int target = 6;
boolean found = false;
for (int i = 0; i < arr.length; i++) {
    for (int j = 0; j < arr[i].length; j++) {
        for (int k = 0; k < arr[i][j].length; k++)
        {
            if (arr[i][j][k] == target) {
                System.out.println("Found at position: (" + i + ", " + j + ", " + k + ")");
                found = true;
            }
        }
    }
}

if (!found)
    System.out.println("Element not found"); */

```

```

//Q19
/* int[][] jagged = new int[3][];
jagged[0] = new int[2];
jagged[1] = new int[3];
jagged[2] = new int[4];
int value = 1;
for (int i = 0; i < jagged.length; i++) {
    for (int j = 0; j < jagged[i].length; j++) {
        jagged[i][j] = value++;
    }
}

```



```

}
}
System.out.println("Jagged Array:");
for (int[] row : jagged) {
    for (int val : row) {
        System.out.print(val + " ");
    }
    System.out.println();
}
}
*/

```

//Q20

```

/*int[][] marks = new int[3][];

marks[0] = new int[]{85, 90}; // Student 1 (2subjects)
marks[1] = new int[]{75, 80, 95}; // Student 2 (3subjects)
marks[2] = new int[]{60, 70, 65, 88}; // Student 3 (4subjects)

System.out.println("Students Marks:");
for (int i = 0; i < marks.length; i++) {
    System.out.print("Student " + (i+1) + ": ");
    for (int mark : marks[i]) {
        System.out.print(mark + " ");
    }
    System.out.println();
}
}
*/

```

//Q21

```

/*int arr[] = {1, 2, 3, 4, 5};

    int first = arr[0];
    for (int i = 0; i < arr.length - 1; i++) {
        arr[i] = arr[i + 1];
    }
}

```

```

arr[arr.length - 1] = first;

System.out.print("Array after left rotation: ");

for (int x : arr) {
    System.out.print(x + " ");
} */

```

//Q22

```

/*int arr[] = {1, 2, 2, 3, 4, 1, 5, 2};
javac boolean visited[] = new boolean[arr.length];
    for (int i = 0; i < arr.length; i++) {
        if (visited[i]) continue;
        int count = 1;
        for (int j = i + 1; j < arr.length; j++) {
            if (arr[i] == arr[j]) {
                count++;
                visited[j] = true;
            }
        }
        System.out.println(arr[i] + " occurs " + count + " times");
    } */

```

//Q23

```

/*int arr[] = {12, 35, 1, 10, 34, 1};
int first = Integer.MIN_VALUE, second = Integer.MIN_VALUE;

for (int n : arr) {
    if (n > first) {
        second = first;
        first = n;
    } else if (n > second && n != first) {

```

```

        second = n;
    }
}

System.out.println("Second Largest Element: " + second); */

```

//Q24

```

/*int arr[] = {1, 2, 3, 2, 4, 3, 5};
int n = arr.length;

for (int i = 0; i < n; i++) {
    for (int j = i + 1; j < n; j++) {
        if (arr[i] == arr[j]) {
            for (int k = j; k < n - 1; k++) {
                arr[k] = arr[k + 1];
            }
            n--;
        } else {
            j++;
        }
    }
}

System.out.print("Array without duplicates: ");
for (int i = 0; i < n; i++) {
    System.out.print(arr[i] + " ");
} */

```

//Q25

```

/* int mat[][] = {
    {1, 2, 3},

```

```
    {4, 5, 6},  
    {7, 8, 9}  
};
```

```
int n = mat.length;  
int mainSum = 0, secSum = 0;
```

```
for (int i = 0; i < n; i++) {  
    mainSum += mat[i][i];  
    secSum += mat[i][n - i - 1];  
}
```

```
System.out.println("Main diagonal sum: " + mainSum);  
System.out.println("Secondary diagonal sum: " + secSum); */
```

```
//Q26
```

```
/*int mat[][] = { {1, 2, 3}, {4, 5, 6}, {7, 8, 9} };
```

```
int n = mat.length;
```

```
System.out.println("Upper Triangular:");
```

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n; j++) {  
        if (i <= j)  
            System.out.print(mat[i][j] + " ");  
        else  
            System.out.print("0 ");  
    }  
    System.out.println();  
}
```

```
System.out.println();
```

```
}
```

```
System.out.println("\nLower Triangular:");
```

```
for (int i = 0; i < n; i++) {
```

```

        for (int j = 0; j < n; j++) {
            if (i >= j)
                System.out.print(mat[i][j] + " ");
            else
                System.out.print("0 ");
        }
        System.out.println();
    }
}
*/

```

//Q27

```

/*int mat[][] = {{1, 2, 3, 4},{5, 6, 7, 8},{9, 10, 11, 12}, {13, 14, 15, 16} };

int rows = mat.length, cols = mat[0].length;

for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        if (i == 0 || i == rows - 1 || j == 0 || j == cols - 1)
            System.out.print(mat[i][j] + " ");
        else
            System.out.print(" ");
    }
    System.out.println();
}
*/

```

//Q28

```

/*int mat[][]= {{10,20,30} , {40,50,60}, {70,80,90} };

int target=50;

boolean found= false;

for(int i=0; i<mat.length;i++)
{
    for(int j=0;j<mat[i].length;j++){
        if(mat[i][j] == target)

```

```

        System.out.println("Element " +target+ " found at Row " +i+", Column" +j);
        found=true;
    }
}
System.out.println("Element not found."); */

```

//Q29

```

/* int arr[][] = {
    {5, 2, 9},
    {1, 7},
    {4, 6, 8, 3}
};
for (int i = 0; i < arr.length; i++) {
    Array.sort(arr[i]);
}
System.out.println("Jagged Array after sorting:");
for (int[] row : arr) {
    for (int x : row) {
        System.out.print(x + " ");
    }
    System.out.println();
} */

```

//Q30

```

int mat[][] = {
    {0, 0, 3},
    {0, 0, 0},
    {4, 0, 0}
};
int rows = mat.length, cols = mat[0].length;

```

```
int total = rows * cols, zeroCount = 0;
```

```
for (int[] row : mat) {
```

```
    for (int val : row) {
```

```
        if (val == 0) zeroCount++;
```

```
    }
```

```
}
```

```
if (zeroCount > total / 2)
```

```
    System.out.println("It is a Sparse Matrix");
```

```
else
```

```
    System.out.println("It is not a Sparse Matrix");
```

```
}
```

```
}
```

Labsheet-4

1) Student Class

```
class Student {
```

```
String name;
```

```
int age;
```

```
static int count = 0;
```

```
void setDetails() {
```

```
String localName = "Rahul"; // local variable
```

```
int localAge = 20; // local variable
```

```
name = localName;
```

```
age = localAge;
```

```
count++;
```

```
}
```

```
void display() {
```

```
System.out.println("Name: " + name + ", Age: " + age + ", Total
```

```
Students: " + count);
```

```
}
```

```
public static void main(String[] args) {
```

```
Student s1 = new Student();
```

```
s1.setDetails();
```

```
s1.display();
```

```
}
```

```
}
```

2) Rectangle Class

```
class Rectangle {
```

```
double length;
```

```
double breadth;
```

```
static String shapeName = "Rectangle";
```

```
void calculateArea() {
```

```
double l = 10; // local variable
```



```

double b = 5; // local variable
length = l;
breadth = b;
double area = length * breadth; // local variable
System.out.println("Shape: " + shapeName + ", Area: " + area);
}
public static void main(String[] args) {
Rectangle rect = new Rectangle();
rect.calculateArea();
}
}

```

3) Circle Class

```

class Circle {
double radius;
static double PI = 3.14;
void calculate() {
double r = 7; // local variable
radius = r;
double area = PI * r * r; // local variable
double circumference = 2 * PI * r; // local variable
System.out.println("Radius: " + radius + ", Area: " + area + ",
Circumference: " + circumference);
}
public static void main(String[] args) {
Circle c = new Circle();
c.calculate();
}
}

```

4) Employee Class

```
class Employee {  
    int empId;  
    double salary;  
    static String companyName = "TechCorp";  
    void setDetails() {  
        int id = 101; // local variable  
        double sal = 55000; // local variable  
        empId = id;  
        salary = sal;  
        System.out.println("Employee ID: " + empId + ", Salary: " + salary + ",  
        Company: " + companyName);  
    }  
    public static void main(String[] args) {  
        Employee e = new Employee();  
        e.setDetails();  
    }  
}
```

5) Book Class

```
class Book {  
    String title;  
    String author;  
    double price;  
    static String publisher = "Pearson";  
    void setDetails() {  
        String t = "Java Basics"; // local variable  
        String a = "John Smith"; // local variable  
        double p = 299.99; // local variable  
        title = t;  
        author = a;  
        price = p;  
    }  
}
```

```

System.out.println("Book: " + title + ", Author: " + author + ", Price:
" + price + ", Publisher: " + publisher);
}
public static void main(String[] args) {
    Book b = new Book();
    b.setDetails();
}
}

```

6) Car Class

```

class Car {
    String brand;
    double mileage;
    static int wheels = 4;
    void display() {
        String b = "Toyota"; // local variable
        double m = 18.5; // local variable
        brand = b;
        mileage = m;
        System.out.println("Brand: " + brand + ", Mileage: " + mileage + " kmpl,
        Wheels: " + wheels);
    }
    public static void main(String[] args) {
        Car c = new Car();
        c.display();
    }
}

```

7) BankAccount Class

```

class BankAccount {
    int accountNumber;

```

```

double balance;
static String bankName = "SBI";
void deposit() {
int acc = 12345; // local variable
double bal = 5000; // local variable
double depositAmount = 2000; // local variable
accountNumber = acc;
balance = bal + depositAmount;
System.out.println("Account No: " + accountNumber + ", Balance: " +
balance + ", Bank: " + bankName);
}
public static void main(String[] args) {
BankAccount ba = new BankAccount();
ba.deposit();
}
}

```

8) Pen Class

```

class Pen {
String color;
String type;
static String manufacturer = "Cello";
void display() {
String c = "Blue"; // local variable
String t = "Gel"; // local variable
color = c;
type = t;
System.out.println("Pen Color: " + color + ", Type: " + type + ",
Manufacturer: " + manufacturer);
}
}

```

```
public static void main(String[] args) {  
    Pen p = new Pen();  
    p.display();  
}  
}
```

9) Laptop Class

```
class Laptop {  
    String brand;  
    int RAM;  
    static String os = "Windows";  
    void display() {  
        String b = "Dell"; // local variable  
        int r = 16; // local variable  
        brand = b;  
        RAM = r;  
        System.out.println("Laptop Brand: " + brand + ", RAM: " + RAM + "GB, OS:  
" + os);  
    }  
    public static void main(String[] args) {  
        Laptop l = new Laptop();  
        l.display();  
    }  
}
```

10) Movie Class

```
class Movie {  
    String name;  
    String genre;  
    double rating;  
    static String industry = "Bollywood";
```

```

void display() {
String n = "3 Idiots"; // local variable
String g = "Comedy"; // local variable
double r = 9.0; // local variable

name = n;
genre = g;
rating = r;

System.out.println("Movie: " + name + ", Genre: " + genre + ", Rating: "
+ rating + ", Industry: " + industry);
}

public static void main(String[] args) {
Movie m = new Movie();
m.display();
}
}

```

Intermediate Level (Q11–Q20)

Focus: encapsulation, conditions, multiple methods, local variables, and class variables.

1) Calculator

```

class Calculator {
static int operationsCount = 0;

void add() {
int a = 10; // local variable
int b = 20;
int result = a + b;
operationsCount++;

System.out.println("Addition: " + result + ", Total Operations: " +
operationsCount);
}

void subtract() {

```

```
int a = 30;

int b = 12;
int result = a - b;
operationsCount++;
System.out.println("Subtraction: " + result + ", Total Operations: " +
operationsCount);
}

void multiply() {
int a = 5;
int b = 6;
int result = a * b;
operationsCount++;
System.out.println("Multiplication: " + result + ", Total Operations: "
+ operationsCount);
}

void divide() {
int a = 40;
int b = 8;
int result = a / b;
operationsCount++;
System.out.println("Division: " + result + ", Total Operations: " +
operationsCount);
}

public static void main(String[] args) {
Calculator c = new Calculator();
c.add();
c.subtract();
c.multiply();
c.divide();
}
```

```
}
```

```
}
```

2) StudentMarks

```
class StudentMarks {
```

```
int marks1, marks2, marks3;
```

```
static String universityName = "Delhi University";
```

```
void calculateAverage() {
```

```
int m1 = 85; // local variable
```

```
int m2 = 90;
```

```
int m3 = 80;
```

```
marks1 = m1;
```

```
marks2 = m2;
```

```
marks3 = m3;
```

```
double avg = (marks1 + marks2 + marks3) / 3.0; // local variable
```

```
System.out.println("University: " + universityName + ", Average Marks: "
```

```
+ avg);
```

```
}
```

```
public static void main(String[] args) {
```

```
StudentMarks s = new StudentMarks();
```

```
s.calculateAverage();
```

```
}
```

```
}
```

3) Temperature

```
class Temperature {
```

```
double celsius;
```

```
static String scale = "Celsius";
```

```
void convert() {
```

```
double c = 25; // local variable
```

```
celsius = c;
```



```

double fahrenheit = (c * 9/5) + 32;
double kelvin = c + 273.15;
System.out.println("Temp in " + scale + ": " + celsius +
", Fahrenheit: " + fahrenheit +
", Kelvin: " + kelvin);
}
public static void main(String[] args) {
Temperature t = new Temperature();
t.convert();
}
}

```

4) Person

```

class Person {
String name;
int age;
static String country = "India";
void checkVotingEligibility() {
String n = "Ravi"; // local variable
int a = 17; // local variable
name = n;
age = a;
if (age >= 18) {

System.out.println(name + " from " + country + " is eligible to
vote.");
} else {
System.out.println(name + " from " + country + " is NOT eligible to
vote.");
}
}
}

```

```

public static void main(String[] args) {
    Person p = new Person();
    p.checkVotingEligibility();
}
}

```

5) SimpleInterest

```

class SimpleInterest {
    double principal, rate, time;
    static String bank = "HDFC Bank";
    void calculate() {
        double p = 10000; // local variable
        double r = 5;
        double t = 2;
        principal = p;
        rate = r;
        time = t;
        double si = (p * r * t) / 100;
        System.out.println("Bank: " + bank + ", Simple Interest: " + si);
    }
    public static void main(String[] args) {
        SimpleInterest s = new SimpleInterest();
        s.calculate();
    }
}

```

6) Triangle

```

class Triangle {
    double a, b, c;
    static String type;
    void checkValidity() {
        double x = 3, y = 4, z = 5; // local variables
    }
}

```

```

a = x; b = y; c = z;
if (a + b > c && a + c > b && b + c > a) {

    if (a == b && b == c) type = "Equilateral";
    else if (a == b || b == c || a == c) type = "Isosceles";
    else type = "Scalene";
    System.out.println("Valid Triangle, Type: " + type);
} else {
    System.out.println("Not a valid Triangle");
}
}

public static void main(String[] args) {
    Triangle t = new Triangle();
    t.checkValidity();
}
}

```

7) StudentGrade

```

class StudentGrade {
    String name;
    int marks;
    static int passingMarks = 40;
    void assignGrade() {
        String n = "Anita"; // local variable
        int m = 78; // local variable
        name = n;
        marks = m;
        char grade;
        if (marks >= 90) grade = 'A';
        else if (marks >= 75) grade = 'B';
        else if (marks >= 60) grade = 'C';
    }
}

```

```

else if (marks >= passingMarks) grade = 'D';
else grade = 'F';
System.out.println("Student: " + name + ", Marks: " + marks + ", Grade:
" + grade);
}
public static void main(String[] args) {
StudentGrade sg = new StudentGrade();
sg.assignGrade();
}
}

```

8) ElectricBill

```

class ElectricBill {
int units;
static double fixedCharge = 50.0;
void calculateBill() {
int u = 120; // local variable
units = u;
double bill;
if (u <= 100) bill = u * 5;
else bill = (100 * 5) + (u - 100) * 7;
double total = bill + fixedCharge;
System.out.println("Units: " + units + ", Total Bill: " + total);
}
public static void main(String[] args) {
ElectricBill eb = new ElectricBill();
eb.calculateBill();
}
}

```

9) Product

```

class Product {
    int id;
    String name;
    double price;
    static double discountRate = 10; // %
    void calculateFinalPrice() {
        int pid = 101; // local variable
        String pname = "Laptop";
        double p = 50000;
        id = pid;
        name = pname;
        price = p;
        double discount = (price * discountRate) / 100;
        double finalPrice = price - discount;
        System.out.println("Product: " + name + ", Original Price: " + price +
            ", Final Price after " + discountRate + "% discount:
            " + finalPrice);
    }
}

```

```

public static void main(String[] args) {
    Product pr = new Product();
    pr.calculateFinalPrice();
}
}

```

10) BankAccount

```

class BankAccount {
    double balance;
    static String bankCode = "ICICI123";
    void withdraw() {
        double bal = 8000; // local variable
    }
}

```

```

double withdrawAmt = 9000;
balance = bal;
if (withdrawAmt <= balance) {
    balance -= withdrawAmt;
    System.out.println("Withdrawal successful. Remaining Balance: " +
        balance + ", Bank Code: " + bankCode);
} else {
    System.out.println("Insufficient Balance! Current Balance: " +
        balance);
}
}

public static void main(String[] args) {
    BankAccount ba = new BankAccount();
    ba.withdraw();
}
}

```

Advanced Level (Q21–Q30)

Focus: problem solving, algorithms, object comparisons, static counters, local variables.

1) Palindrome

```

class Palindrome {
    static int countChecks = 0;
    void checkPalindrome() {
        int num = 121; // local variable
        int original = num; // local variable
        int rev = 0;
        while (num > 0) {
            int digit = num % 10;
            rev = rev * 10 + digit;

```

```

num /= 10;
}
countChecks++;
if (original == rev) {
System.out.println(original + " is a Palindrome. Checks: " +
countChecks);
} else {
System.out.println(original + " is NOT a Palindrome. Checks: " +
countChecks);
}
}

public static void main(String[] args) {
Palindrome p = new Palindrome();
p.checkPalindrome();
}
}

```

2) Armstrong

```

class Armstrong {
static int totalChecks = 0;
void checkArmstrong() {
int num = 153; // local variable
int original = num;
int sum = 0;
while (num > 0) {
int digit = num % 10;

sum += digit * digit * digit;
num /= 10;
}
totalChecks++;
}
}

```

```

if (original == sum) {
    System.out.println(original + " is an Armstrong Number. Total
Checks: " + totalChecks);
} else {
    System.out.println(original + " is NOT an Armstrong Number. Total
Checks: " + totalChecks);
}
}
}

public static void main(String[] args) {
    Armstrong a = new Armstrong();
    a.checkArmstrong();
}
}

```

3) PrimeChecker

```

class PrimeChecker {
    static int totalPrimeChecks = 0;
    void checkPrime() {
        int num = 29; // local variable
        boolean isPrime = true;
        if (num <= 1) isPrime = false;
        else {
            for (int i = 2; i <= num / 2; i++) {
                if (num % i == 0) {
                    isPrime = false;
                    break;
                }
            }
        }
        totalPrimeChecks++;
        if (isPrime) {

```



```

System.out.println(num + " is Prime. Total Checks: " +
totalPrimeChecks);
} else {
System.out.println(num + " is NOT Prime. Total Checks: " +
totalPrimeChecks);
}
}

```

```

public static void main(String[] args) {
PrimeChecker pc = new PrimeChecker();
pc.checkPrime();
}
}

```

4) Fibonacci

```

class Fibonacci {
static int seriesCount = 0;
void generateSeries() {
int n = 7; // local variable
int a = 0, b = 1;
System.out.print("Fibonacci Series: ");
for (int i = 1; i <= n; i++) {
System.out.print(a + " ");
int next = a + b;
a = b;
b = next;
seriesCount++;
}
System.out.println("\nTotal Terms Generated: " + seriesCount);
}
public static void main(String[] args) {

```

```
Fibonacci f = new Fibonacci();
```

```
f.generateSeries();
```

```
}
```

```
}
```

5) Factorial

```
class Factorial {
```

```
static int totalCalls = 0;
```

```
void computeFactorial() {
```

```
int num = 5; // local variable
```

```
int fact = 1;
```

```
for (int i = 1; i <= num; i++) {
```

```
fact *= i;
```

```
}
```

```
totalCalls++;
```

```
System.out.println("Factorial of " + num + " is " + fact + ". Total
```

```
Calls: " + totalCalls);
```

```
}
```

```
public static void main(String[] args) {
```

```
Factorial f = new Factorial();
```

```
f.computeFactorial();
```

```
}
```

```
}
```

6) Matrix (2x2)

```
class Matrix {
```

```
int[][] matrix1 = new int[2][2];
```

```
int[][] matrix2 = new int[2][2];
```

```
static String matrixType = "2x2";
```

```
void addSubtract() {
```

```
int[][] m1 = { {1, 2}, {3, 4} }; // local variable
```

```

int[][] m2 = { {5, 6}, {7, 8} }; // local variable
matrix1 = m1;
matrix2 = m2;
int[][] sum = new int[2][2];
int[][] diff = new int[2][2];
for (int i = 0; i < 2; i++) {
    for (int j = 0; j < 2; j++) {
        sum[i][j] = matrix1[i][j] + matrix2[i][j];
        diff[i][j] = matrix1[i][j] - matrix2[i][j];
    }
}

System.out.println("Matrix Type: " + matrixType);
System.out.println("Addition Result: ");
for (int[] row : sum) System.out.println(row[0] + " " + row[1]);
System.out.println("Subtraction Result: ");
for (int[] row : diff) System.out.println(row[0] + " " + row[1]);
}

public static void main(String[] args) {
    Matrix m = new Matrix();
    m.addSubtract();
}
}

```

7) StringReverse

```

class StringReverse {
    static int totalReversals = 0;
    void reverseString() {
        String str = "Hello"; // local variable
        String rev = "";
        for (int i = str.length() - 1; i >= 0; i--) {

```

```

rev += str.charAt(i);
}
totalReversals++;
System.out.println("Original: " + str + ", Reversed: " + rev + ", Total
Reversals: " + totalReversals);
}
public static void main(String[] args) {
StringReverse sr = new StringReverse();
sr.reverseString();
}
}

```

8) WordCounter

```

class WordCounter {
static String language = "English";
void countWords() {
String sentence = "Java is fun to learn"; // local variable
String[] words = sentence.split(" ");
int count = words.length;
System.out.println("Sentence: \"\" + sentence + "\" has " + count + "
words. Language: " + language);
}
public static void main(String[] args) {
WordCounter wc = new WordCounter();
wc.countWords();
}
}

```

9) Employee

```

class Employee {
String name;

```

```

double salary;
static String organization = "Infosys";
void compareSalaries() {
String n1 = "Amit"; // local variables
double s1 = 50000;
String n2 = "Rohit";
double s2 = 60000;
if (s1 > s2) {
System.out.println(n1 + " has higher salary in " + organization);
} else if (s2 > s1) {
System.out.println(n2 + " has higher salary in " + organization);
} else {
System.out.println("Both have equal salary in " + organization);
}
}
public static void main(String[] args) {
Employee e = new Employee();
e.compareSalaries();
}
}

```

10) Library

```

class Library {
int booksAvailable;
static String libraryName = "City Library";
void updateOperations() {
int initialBooks = 100; // local variable
int issue = 5;
int returned = 3;
booksAvailable = initialBooks - issue + returned;
System.out.println("Library: " + libraryName +

```

```
" , Books Available after Issue/Return: " +  
booksAvailable);  
}  
public static void main(String[] args) {  
    Library lib = new Library();  
    lib.updateOperations();  
  
}  
}
```

Labsheet-5

```
class Prog5{  
    //1  
    /*String name;  
    int age;  
    Prog5(String n, int a) {  
        name=n;  
        age=a;  
    }  
    void display(){  
        System.out.println("Name: " +name+ ", Age: "+age);  
    }  
    public static void main(String[] args) {  
        Prog5 s1= new Prog5("Teju",20);  
        Prog5 s2= new Prog5("Tejasvi" , 20);  
        s1.display();  
        s2.display();  
    }*/  
  
    //2  
    /* String title;  
    static int countBooks = 0;  
    Prog5(String t) {  
        title = t;  
        countBooks++;  
    }  
    void show() {  
        System.out.println("Book Title: " + title);  
    }  
}
```

```
public static void main(String[] args) {  
    Prog5 b1 = new Prog5("Java Programming");  
    Prog5 b2 = new Prog5("Python Basics");  
    b1.show();  
    b2.show();  
    System.out.println("Total Books: " + countBooks); */
```

```
//3
```

```
/*double radius;  
Prog5(double r) {  
    radius = r;  
}  
void area() {  
    double area = 3.14 * radius * radius;  
    System.out.println("Area: " + area);  
}  
public static void main(String[] args) {  
    Prog5 c1 = new Prog5(5.0);  
    c1.area(); */
```

```
//4
```

```
/*static int count = 0;  
Prog5() {  
    count++;  
}  
void show() {  
    System.out.println("Objects created: " + count);  
}  
public static void main(String[] args) {  
    Prog5 c1 = new Prog5();
```



```

Prog5 c2 = new Prog5();
Prog5 c3 = new Prog5();
c1.show(); */

//5
/*String empName;
double salary;
Prog5(String n, double s) {
empName = n;
salary = s;
}
void display() {
String msg = "Employee: " + empName + ", Salary: " + salary;
System.out.println(msg);
}
public static void main(String[] args) {
Prog5 e1 = new Prog5("Teju", 40000);
Prog5 e2 = new Prog5("Tejasvi", 50000);
e1.display();
e2.display(); */

//6
/*int length, width;
Prog5(int l, int w) {
length = l;
width = w;
}
void area() {
int area = length * width;
System.out.println("Area: " + area);

```

```

}

public static void main(String[] args) {
    Prog5 r1 = new Prog5(5, 10);
    r1.area(); */

//7
/*String model;
int year;
static int totalCars = 0;
Prog5(String m, int y) {
    model = m;
    year = y;
    totalCars++;
}
void show() {
    System.out.println(model + " - " + year);
}
public static void main(String[] args) {
    Prog5 c1 = new Prog5("Honda Civic", 2021);
    Prog5 c2 = new Prog5("Toyota Corolla", 2020);
    c1.show();
    c2.show();
    System.out.println("Total Cars: " + totalCars); */

//8
/*int height, width , depth;
Prog5(int h, int w, int d) {
    height= h;
    width=w;
    depth=d;

```

```

    }

    void volume(){
        int vol= height*width*depth;
        System.out.println("Volume: "+ vol);
    }

    public static void main(String[]s){
        Prog5 p=new Prog5(2,3,4);
        p.volume();
    }*/

//9
/*String brand;
Prog5(String b) {
    brand= b;
}

void show(){
    int price= 15000;
    System.out.println("Brand: "+brand+ ", Price: " +price);
}

public static void main(String[] args) {
    Prog5 p1= new Prog5("Samsung");
    Prog5 p2= new Prog5("Apple");
    p1.show();
    p2.show();
}*/

//10
/*int accountNumber;
double balance;
Prog5(int acc, double bal){

```

```

        accountNumber=acc;
        balance=bal;
    }
    void deposit(double amount){
        balance += amount;
        System.out.println("Updated Balance: "+balance);
    }
    public static void main(String[] args) {
        Prog5 p= new Prog5(101, 5000);
        p.deposit(2000);
    }*/

//21
/* String title;
String author;
double price;
Prog5(String title, String author, double price) {
    this.title = title;
    this.author = author;
    this.price = price;
}
Prog5(String title, String author) {
    this(title, author, 500.0);
}
Prog5() {
    this("Unknown", "Unknown", 0.0);
}
void display() {
    System.out.println("Title: " + title + ", Author: " + author + ", Price:" + price);
}

```

```
public static void main(String[] args) {  
    Prog5 b1 = new Prog5("Java Basics", "Herbert", 600);  
    Prog5 b2 = new Prog5("C Programming", "Dennis");  
    Prog5 b3 = new Prog5();  
    b1.display();  
    b2.display();  
    b3.display(); */
```

```
//22
```

```
/*int length, breadth;
```

```
Prog5() {
```

```
    this.length = 1;
```

```
    this.breadth = 1;
```

```
}
```

```
Prog5(int side) {
```

```
    this.length = side;
```

```
    this.breadth = side;
```

```
}
```

```
Prog5(int length, int breadth) {
```

```
    this.length = length;
```

```
    this.breadth = breadth;
```

```
}
```

```
int area() {
```

```
    return length * breadth;
```

```
}
```

```
public static void main(String[] args) {
```

```
    Prog5 r1 = new Prog5();
```

```
    Prog5 r2 = new Prog5(5);
```

```
    Prog5 r3 = new Prog5(4, 6);
```

```
    System.out.println("Area (default): " + r1.area());
```

```

System.out.println("Area (square): " + r2.area());
System.out.println("Area (rectangle): " + r3.area()); */

//23
/*int id;
String name;
double salary;
Prog5(int id, String name, double salary) {
this.id = id;
this.name = name;
this.salary = salary;
}
Prog5(int id, String name) {
this(id, name, 30000.0);
}
void display() {
System.out.println("ID: " + id + ", Name: " + name + ", Salary: " +
salary);
}
public static void main(String[] args) {
Prog5 e1 = new Prog5(101, "Teju", 45000);
Prog5 e2 = new Prog5(102, "Tejasvi");
e1.display();
e2.display(); */

//24
/*String brand;
String model;
int year;
Prog5(String brand) {

```

```

this.brand = brand;
this.model = "Unknown";
this.year = 0;
}

Prog5(String brand, String model) {
this.brand = brand;
this.model = model;
this.year = 0;
}

Prog5(String brand, String model, int year) {
this.brand = brand;
this.model = model;
this.year = year;
}

void display() {
System.out.println("Car: " + brand + ", Model: " + model + ", Year:" + year);
}

public static void main(String[] args) {
Prog5 c1 = new Prog5("Toyota");
Prog5 c2 = new Prog5("Honda", "Civic");
Prog5 c3 = new Prog5("Ford", "Mustang", 2022);
c1.display();
c2.display();
c3.display();*/

//25
/*String name;
int rollNo;
Prog5(String name) {
this.name = name;

```

```

this.rollNo = -1;
}
Prog5(String name, int rollNo) {
this.name = name;
this.rollNo = rollNo;
}
void display() {
System.out.println("Name: " + name + ", Roll No: " + rollNo);
}
public static void main(String[] args) {
Prog5 s1 = new Prog5("Teju");
Prog5 s2 = new Prog5("Tejasvi", 101);
s1.display();
s2.display(); */

//26
/*int add(int a, int b) {
return a + b;
}
double add(double a, double b) {
return a + b;
}
String add(String a, String b) {
return a + b;
}
public static void main(String[] args) {
Prog5 calc = new Prog5();
System.out.println("Sum (int): " + calc.add(5, 7));
System.out.println("Sum (double): " + calc.add(4.5, 3.2));
System.out.println("Sum (string): " + calc.add("Hello ", "World")); */

```


//27

```
/*void print(int num) {  
System.out.println("Integer: " + num);  
}  
void print(double num) {  
System.out.println("Double: " + num);  
}  
void print(String text) {  
System.out.println("String: " + text);  
}  
public static void main(String[] args) {  
Prog5 p = new Prog5();  
p.print(100);  
p.print(45.67);  
p.print("Overloading Example"); */
```

//28

```
/*int multiply(int a, int b) {  
return a * b;  
}  
int multiply(int a, int b, int c) {  
return a * b * c;  
}  
double multiply(double a, double b) {  
return a * b;  
}  
public static void main(String[] args) {  
Prog5 mo = new Prog5();  
System.out.println("Multiply (2 int): " + mo.multiply(3, 4));
```

```

System.out.println("Multiply (3 int): " + mo.multiply(2, 3, 4));
System.out.println("Multiply (double): " + mo.multiply(2.5, 4.0)); */

//30

/*int area(int side) { // square
return side * side;
}

int area(int length, int breadth) { // rectangle
return length * breadth;
}

double area(double radius) { // circle
return 3.14159 * radius * radius;
}

public static void main(String[] args) {
Prog5 sa = new Prog5();
System.out.println("Square area: " + sa.area(5));
System.out.println("Rectangle area: " + sa.area(4, 6));
System.out.println("Circle area: " + sa.area(3.5)); */

//11

/* static String bankName = "ABC Bank";
int accountNumber;
double balance;
Prog5(int accNo, double bal) {
    accountNumber = accNo;
    balance = bal;
}

void deposit(double amount) {
    balance += amount;
    System.out.println("Deposited: " + amount + ", New Balance: " + balance);
}

```

```

    }

    void display() {
        System.out.println("Bank: " + bankName + ", Account: " + accountNumber + ",
Balance: " + balance);
    }

    public static void main(String[] args) {
        Prog5 a1 = new Prog5(101, 5000);
        a1.deposit(2000);
        a1.display(); */

//12
/* String courseName;
int duration;
static int totalCourses = 0;
Prog5(String name, int dur) {
    courseName = name;
    duration = dur;
    totalCourses++;
}

void display() {
    System.out.println("Course: " + courseName + ", Duration: " + duration + " months");
    System.out.println("Total Courses: " + totalCourses);
}

public static void main(String[] args) {
    Prog5 c1 = new Prog5("Java", 6);
    Prog5 c2 = new Prog5("Python", 4);
    c1.display();
    c2.display(); */

//13

```

```

/* String itemName;

int quantity;

static int totalItems = 0;

Prog5(String name, int qty) {
    itemName = name;
    quantity = qty;
    totalItems += qty;
}

void display() {
    System.out.println("Item: " + itemName + ", Quantity: " + quantity);
    System.out.println("Total Items: " + totalItems);
}

public static void main(String[] args) {
    Prog5 i1 = new Prog5("Laptops", 10);
    Prog5 i2 = new Prog5("Mobiles", 20);
    i1.display();
    i2.display(); */

//14

/* String cityName;

static String countryName = "India";

Prog5(String name) {
    cityName = name;
}

void showDetails() {
    int population = 1000000;

    System.out.println("City: " + cityName + ", Country: " + countryName + ", Population:
" + population);
}

```

```

public static void main(String[] args) {
    Prog5 c1 = new Prog5("Delhi");
    c1.showDetails(); */

    //15
    /* String laptopName;
    double price;
    static String storeName = "Tech World";
    Prog5(String name, double p) {
        laptopName = name;
        price = p;
    }
    void applyDiscount() {
        double discount = 0.1 * price;
        double finalPrice = price - discount;
        System.out.println("Laptop: " + laptopName + ", Price after discount: " + finalPrice);
    }
    public static void main(String[] args) {
        Prog5 l1 = new Prog5("Dell XPS", 80000);
        l1.applyDiscount(); */

    //16
    /* String roomType;
    static String hotelName = "Grand Palace";
    Prog5(String type) {
        roomType = type;
    }
    void bookingDetails() {
        System.out.println("Hotel: " + hotelName + ", Room Type: " + roomType);
    }

```

```

public static void main(String[] args) {
    Prog5 h1 = new Prog5("Deluxe");
    h1.bookingDetails(); */

    //17
    /* String studentName;
    int rollNumber;
    static String universityName = "Global University";

    Prog5(String name, int roll) {
        studentName = name;
        rollNumber = roll;
    }
    void display() {
        System.out.println("Student: " + studentName + ", Roll: " + rollNumber + ", University:
" + universityName);
    }
    public static void main(String[] args) {
        Prog5 u1 = new Prog5("Tejasvi", 101);
        u1.display(); */

        //18
        /*int flightNumber;
        String destination;
        static String airlineName = "Sky Airlines";
        Prog5(int number, String dest) {
            flightNumber = number;
            destination = dest;
        }
        void showDetails() {

```

```
        System.out.println("Flight No: " + flightNumber + ", Destination: " + destination + ",  
Airline: " + airlineName);
```

```
    }
```

```
    public static void main(String[] args) {
```

```
        Prog5 f1 = new Prog5(12345, "London");
```

```
        f1.showDetails(); */
```

```
        //19
```

```
        /*String gameName;
```

```
        String genre;
```

```
        static String platform = "PC";
```

```
        Prog5(String name, String gen) {
```

```
            gameName = name;
```

```
            genre = gen;
```

```
        }
```

```
        void showDetails() {
```

```
            double rating = 4.5;
```

```
            System.out.println("Game: " + gameName + ", Genre: " + genre + ", Platform: " +  
platform + ", Rating: " + rating);
```

```
        }
```

```
    public static void main(String[] args) {
```

```
        Prog5 g1 = new Prog5("Minecraft", "Sandbox");
```

```
        g1.showDetails(); */
```

```
        //20
```

```
        String itemName;
```

```
        static String storeLocation = "Downtown";
```

```
        Prog5(String name) {
```

```
            itemName = name;
```

```
        }
```

```
        void finalPrice(double price) {
```

```
        double discount = 0.2 * price;

        double finalPrice = price - discount;

        System.out.println("Item: " + itemName + ", Store: " + storeLocation + ", Final Price: "
+ finalPrice);

    }

    public static void main(String[] args) {
Prog5 s1 = new Prog5("Smartphone");
        s1.finalPrice(30000);

    }

}
```


Labsheet-6

```
//1
class Animal{
    void sound(){
        System.out.println("Some generic animal sound.");
    }
}

class Dog extends Animal{
    void sound(){
        System.out.println("Bark");
    }
}

public class Prog6{
    public static void main(String s[]){
        Dog d= new Dog();
        d.sound();
    }
}
```

```
//2
/* class Person{
    String name;
    Person(String name){
        this.name=name;
    }
}

class Student extends Person{
    Student(String name){
        super(name);
    }
}
```

```

        System.out.println("Student name: "+ this.name);

    }

}

public class Main1{
    public static void main(String s[]){
        Student s1= new Student("Tejasvi");

    }
} */

//3
/*class Shape{
    void area(){
        System.out.println("Calculating area.");
    }
}

class Circle extends Shape{
    double radius;
    Circle(double radius){
        this.radius=radius;
    }
    void area(){
        double result= Math.PI * radius * radius ;
        System.out.println("Area of circle: "+result);
    }
}

public class Prog6{
    public static void main(String[] args) {
        Shape s= new Shape();

```

```

        s.area();

        Circle c= new Circle(5);

        c.area();

    }

}*/

//4

/*class Vehicle{

    void move(){

        System.out.println("Vehicle is moving");

    }

}

class Car extends Vehicle{

}

public class Prog6{

    public static void main(String[] args) {

        Car c=new Car();

        c.move();

    }

}*/

//17

/*interface Printable{

    void print();

}

interface Scannable{

    void scan();

}

class MultiFunctionPrinter implements Printable, Scannable{

```

```

public void print(){
    System.out.println("Printer is printing");
}
public void scan(){
    System.out.println("Scanner is scanning");
}
}
class Prog6{
    public static void main(String[] args) {
        MultiFunctionPrinter m= new MultiFunctionPrinter();
        m.print();
        m.scan();
    }
}*/

//3
/*class Shape{
    void area(){
        System.out.println("Calculating area");
    }
}
class Circle extends Shape{
    double radius;
    Circle(double radius){
        this.radius=radius;
    }
    void area(){
        double result= Math.PI*radius*radius;
        System.out.println("Area of circle: "+ result);
    }
}

```

```

}

class Prog6{
    public static void main(String[] args) {
        Shape s= new Shape();
        Circle c= new Circle(5);
        s.area();
        c.area();
    }
}*/

//5
/*class Employee{
    int id; String name;
    Employee(int id, String name){
        this.id= id; this.name=name;
    }
}

class Manager extends Employee{
    Manager(int id, String name){
        super(id, name);
        System.out.println("Manager id: "+ this.id);
        System.out.println("Manager name: "+ this.name);
    }
}

class Prog6{
    public static void main(String[] args) {
        Manager m= new Manager(101, "Teju");
    }
} */

```

//6

```
/*class Employee{
    String name;

    public Employee(String name) {
        this.name=name;
    }

}

class Manager extends Employee{
    Manager(String name){
        super(name);
        System.out.println("Manager name: "+this.name);
    }
}

class Prog6{
    public static void main(String[] args) {
        Manager m= new Manager("Tejasvi");
    }
}*/
```

//7

```
/*class Book {
    String title, author;

    Book() {
        System.out.println("Book constructor without parameters called");
    }

    Book(String title, String author) {
```

```
        this.title = title;

        this.author = author;

        System.out.println("Book Title: " + title + ", Author: " + author);
    }
}
```

```
class Magazine extends Book {
    Magazine() {
        super();
    }

    Magazine(String title, String author) {
        super(title, author);
    }
}
```

```
public class Prog6{
    public static void main(String[] args) {
        new Magazine();
        new Magazine("Tech World", "John Smith");
    }
}*/
```

```
//8
```

```
/*class Calculator {
    int add(int a, int b) {
        return a + b;
    }

    double add(double a, double b) {
```

```
        return a + b;
    }
}
```

```
class AdvancedCalculator extends Calculator {
    void showResults() {
        System.out.println("Sum of integers: " + add(10, 20));
        System.out.println("Sum of doubles: " + add(10.5, 20.3));
    }
}
```

```
public class Main8 {
    public static void main(String[] args) {
        AdvancedCalculator ac = new AdvancedCalculator();
        ac.showResults();
    }
}*/
```

```
//9
```

```
/*class Employee {
    private void display() {
        System.out.println("Employee display method");
    }
}
```

```
class Manager extends Employee {
    void show() {
        System.out.println("Private methods are not inherited in subclasses");
    }
}
```



```
}
```

```
public class Prog6 {  
    public static void main(String[] args) {  
        Manager m = new Manager();  
        m.show();  
    }  
}*/
```

```
//10
```

```
/*class Car {  
    protected void startEngine() {  
        System.out.println("Car engine started...");  
    }  
}
```

```
class ElectricCar extends Car {  
    void showStart() {  
        startEngine(); // accessing protected method of superclass  
        System.out.println("ElectricCar is running silently...");  
    }  
}
```

```
public class Prog6 {  
    public static void main(String[] args) {  
        ElectricCar ec = new ElectricCar();  
        ec.showStart();  
    }  
}*/
```

//11

```
/*class Animal {  
    void eat() {  
        System.out.println("Animal is eating");  
    }  
}
```

```
class Mammal extends Animal {  
    // Mammal inherits eat() from Animal  
}
```

```
class Dog extends Mammal {  
    @Override  
    void eat() {  
        super.eat(); // calls Animal's eat()  
        System.out.println("Dog is eating");  
    }  
}
```

```
public class Prog6 {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        d.eat();  
    }  
}*/
```

//12

```
/*class Vehicle {
```

```
String vehicleType;
```

```
Vehicle(String type) {  
    this.vehicleType = type;  
    System.out.println("Vehicle constructor: " + vehicleType);  
}  
}
```

```
class Car extends Vehicle {  
    String carType;
```

```
    Car(String vType, String cType) {  
        super(vType); // call Vehicle constructor  
        this.carType = cType;  
        System.out.println("Car constructor: " + carType);  
    }  
}
```

```
class ElectricCar extends Car {  
    ElectricCar(String vType, String cType) {  
        super(vType, cType); // call Car constructor  
        System.out.println("ElectricCar constructor");  
    }  
}
```

```
public class Prog6 {  
    public static void main(String[] args) {  
        ElectricCar ec = new ElectricCar("Four Wheeler", "Sedan");  
    }  
}*/
```

```
//13
/*class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}
```

```
class Mammal extends Animal {
    @Override
    void sound() {
        System.out.println("Growl");
    }
}
```

```
class Dog extends Mammal {
    @Override
    void sound() {
        System.out.println("Bark");
    }
}
```

```
public class Prog6 {
    public static void main(String[] args) {
        Animal a = new Animal();
        Mammal m = new Mammal();
        Dog d = new Dog();

        a.sound(); // Animal
        m.sound(); // Growl
    }
}
```

```

        d.sound(); // Bark
    }
}*/

//14
/*class Vehicle {
    void showType() {
        System.out.println("This is a Vehicle");
    }
}

class Car extends Vehicle {
    void showType() {
        System.out.println("This is a Car");
    }
}

class ElectricCar extends Car {
    void showType() {
        super.showType();    // Calls Car's method
        super.showType();    // still Car
        super.showType();    // but we want Vehicle's method
        // To call Vehicle directly:
        ((Vehicle)this).showType();
    }
}

public class Prog6 {
    public static void main(String[] args) {
        ElectricCar ec = new ElectricCar();
    }
}

```

```
        ec.showType(); // calls Vehicle method inside ElectricCar
    }
}*/
```

```
//15
```

```
/*class Shape {
    void draw() {
        System.out.println("Drawing Shape");
    }
}
```

```
class Circle extends Shape {
    @Override
    void draw() {
        System.out.println("Drawing Circle");
    }
}
```

```
class Square extends Shape {
    @Override
    void draw() {
        System.out.println("Drawing Square");
    }
}
```

```
public class HierarchicalDemo {
    public static void main(String[] args) {
        Shape s1 = new Circle();
        Shape s2 = new Square();
    }
}
```

```

        s1.draw(); // Circle
        s2.draw(); // Square
    }
}*/

//16
/*class Employee {
    int id;
    String name;

    Employee(int id, String name) {
        this.id = id;
        this.name = name;
        System.out.println("Employee Constructor: " + id + " " + name);
    }
}

class FullTimeEmployee extends Employee {
    FullTimeEmployee(int id, String name) {
        super(id, name); // call Employee constructor
        System.out.println("FullTimeEmployee Constructor");
    }
}

class PartTimeEmployee extends Employee {
    PartTimeEmployee(int id, String name) {
        super(id, name); // call Employee constructor
        System.out.println("PartTimeEmployee Constructor");
    }
}

```

```
public class Prog6 {  
    public static void main(String[] args) {  
        FullTimeEmployee fte = new FullTimeEmployee(101, "Alice");  
        PartTimeEmployee pte = new PartTimeEmployee(102, "Bob");  
    }  
}*/
```

```
//17
```

```
/*interface Printable {  
    void print();  
}
```

```
interface Scannable {  
    void scan();  
}
```

```
class MultiFunctionPrinter implements Printable, Scannable {  
    @Override  
    public void print() {  
        System.out.println("Printing document...");  
    }  
}
```

```
    @Override  
    public void scan() {  
        System.out.println("Scanning document...");  
    }  
}
```



```
public class Prog6 {  
    public static void main(String[] args) {  
        MultiFunctionPrinter mfp = new MultiFunctionPrinter();  
        mfp.print();  
        mfp.scan();  
    }  
}*/
```

```
//18
```

```
/*interface A {  
    default void show() {  
        System.out.println("Show from A");  
    }  
}
```

```
interface B {  
    default void show() {  
        System.out.println("Show from B");  
    }  
}
```

```
class C implements A, B {  
    @Override  
    public void show() {  
        // Resolving conflict explicitly  
        A.super.show(); // or B.super.show()  
        System.out.println("Show from C");  
    }  
}
```

```
public class Prog6 {  
    public static void main(String[] args) {  
        C obj = new C();  
        obj.show();  
    }  
}*/
```

```
//19
```

```
/*class Animal {  
    void eat() {  
        System.out.println("Animal is eating");  
    }  
}
```

```
interface Flyable {  
    void fly();  
}
```

```
class Bird extends Animal implements Flyable {  
    @Override  
    public void fly() {  
        System.out.println("Bird is flying");  
    }
```

```
    @Override  
    void eat() {  
        System.out.println("Bird is eating");  
    }
```

```
}
```

```
public class Prog6 {  
    public static void main(String[] args) {  
        Bird b = new Bird();  
        b.eat();  
        b.fly();  
    }  
}*/
```

```
//20
```

```
/*class Vehicle {  
    public void start() {  
        System.out.println("Vehicle started");  
    }  
}
```

```
interface RemoteControl {  
    void start();  
}
```

```
class Car extends Vehicle implements RemoteControl {  
    @Override  
    public void start() {  
        System.out.println("Car started via RemoteControl");  
    }  
}
```

```
public class Prog6 {  
    public static void main(String[] args) {
```

```
        Car car = new Car();
        car.start();
    }
}*/
```

```
//21
```

```
/*class Machine {
    Machine() {
        System.out.println("Machine constructor called");
    }
}
```

```
interface Electrical {
    default void powerOn() {
        System.out.println("Power On through Electrical interface");
    }
}
```

```
class WashingMachine extends Machine implements Electrical {
    WashingMachine() {
        super();
        System.out.println("WashingMachine constructor called");
    }
}
```

```
public class Prog6{
    public static void main(String[] args) {
        WashingMachine wm = new WashingMachine();
        wm.powerOn();
    }
}
```

```
    }  
}*/
```

```
//22
```

```
/*abstract class Shape {  
    abstract double area();  
}
```

```
class Circle extends Shape {  
    double radius;  
    Circle(double r) { this.radius = r; }  
    @Override  
    double area() { return Math.PI * radius * radius; }  
}
```

```
class Rectangle extends Shape {  
    double l, w;  
    Rectangle(double l, double w) { this.l = l; this.w = w; }  
    @Override  
    double area() { return l * w; }  
}
```

```
public class Prog6 {  
    public static void main(String[] args) {  
        Shape s1 = new Circle(5);  
        Shape s2 = new Rectangle(4, 6);  
        System.out.println("Circle area: " + s1.area());  
        System.out.println("Rectangle area: " + s2.area());  
    }  
}
```

```
*/
```

```
//23
```

```
/*interface A {
```

```
    default int calculate(int x, int y) {
```

```
        return x + y;
```

```
    }
```

```
}
```

```
interface B {
```

```
    default int calculate(int x, int y) {
```

```
        return x * y;
```

```
    }
```

```
}
```

```
class Calculator implements A, B {
```

```
    @Override
```

```
    public int calculate(int x, int y) {
```

```
        return A.super.calculate(x, y); // or B.super.calculate(x, y)
```

```
    }
```

```
}
```

```
public class Prog6 {
```

```
    public static void main(String[] args) {
```

```
        Calculator c = new Calculator();
```

```
        System.out.println("Result: " + c.calculate(5, 10));
```

```
    }
```

```
*/
```

//24

```
/*interface Flyable {  
    void fly();  
}
```

```
class Animal {  
    void eat() {  
        System.out.println("Animal is eating");  
    }  
}
```

```
class Bird extends Animal implements Flyable {  
    @Override  
    public void fly() {  
        System.out.println("Bird is flying");  
    }  
}
```

```
public class Prog6yj {  
    public static void main(String[] args) {  
        Bird b = new Bird();  
        b.eat();  
        b.fly();  
    }  
}*/
```

//25

```
/*class Employee {  
    void work() {  
        System.out.println("Employee is working");  
    }  
}
```

```
class Manager extends Employee {  
    @Override  
    void work() {  
        System.out.println("Manager is managing");  
    }  
}
```

```
class Developer extends Employee {  
    @Override  
    void work() {  
        System.out.println("Developer is coding");  
    }  
}
```

```
public class Prog6 {  
    public static void main(String[] args) {  
        Employee e1 = new Manager();  
        Employee e2 = new Developer();  
        e1.work();  
        e2.work();  
    }  
}*/
```


//26

```
/*class Person {  
    Person(String name) {  
        System.out.println("Person: " + name);  
    }  
}
```

```
class Employee extends Person {  
    Employee(int id) {  
        super("DefaultName");  
        System.out.println("Employee ID: " + id);  
    }  
}
```

```
class Manager extends Employee {  
    Manager() {  
        super(101);  
        System.out.println("Manager constructor called");  
    }  
}
```

```
public class Prog6 {  
    public static void main(String[] args) {  
        Manager m = new Manager();  
    }  
}*/
```

//27

```
/*interface A {
```

```
    default void show() { System.out.println("A show"); }  
}
```

```
interface B {  
    default void show() { System.out.println("B show"); }  
}
```

```
class C implements A, B {  
    @Override  
    public void show() {  
        A.super.show(); // Resolving conflict  
        B.super.show();  
        System.out.println("C show");  
    }  
}
```

```
public class Prog6 {  
    public static void main(String[] args) {  
        C obj = new C();  
        obj.show();  
    }  
}*/
```

```
//28
```

```
/*abstract class Vehicle {  
    Vehicle() {  
        System.out.println("Vehicle constructor");  
    }  
    abstract void start();  
}
```

```
}
```

```
class Car extends Vehicle {  
    Car() { super(); }  
    @Override  
    void start() { System.out.println("Car starting..."); }  
}
```

```
class Truck extends Vehicle {  
    Truck() { super(); }  
    @Override  
    void start() { System.out.println("Truck starting..."); }  
}
```

```
public class Prog6 {  
    public static void main(String[] args) {  
        Vehicle v1 = new Car();  
        Vehicle v2 = new Truck();  
        v1.start();  
        v2.start();  
    }  
}*/
```

```
//29
```

```
/*class Employee {  
    private void calculateSalary() {  
        System.out.println("Salary calculated");  
    }  
    protected void accessSalary() {  
        calculateSalary();  
    }  
}
```

```
    }  
}
```

```
class Manager extends Employee {  
    public void showSalary() {  
        accessSalary();  
    }  
}
```

```
public class Prog6 {  
    public static void main(String[] args) {  
        Manager m = new Manager();  
        m.showSalary();  
    }  
}*/
```

```
//30
```

```
/*interface Driveable {  
    void drive();  
}
```

```
class Vehicle {  
    void move() {  
        System.out.println("Vehicle is moving");  
    }  
}
```

```
class ElectricCar extends Vehicle implements Driveable {  
    @Override
```

```
    public void drive() {  
        System.out.println("ElectricCar is driving silently");  
    }  
}
```

```
public class Prog6 {  
    public static void main(String[] args) {  
        ElectricCar e = new ElectricCar();  
        e.move();  
        e.drive();  
    }  
}*/
```

Labsheet-7

Question 1: DivideByZeroDemo

```
class DivideByZeroDemo {  
    public static void main(String[] args) {  
        try {  
            int a = 10, b = 0;  
            int c = a / b;  
            System.out.println(c);  
        } catch (ArithmeticException e) {  
            System.out.println("Exception: " + e.getMessage());  
        }  
    }  
}
```

Question 2: ArrayExceptionExample

```
class ArrayExceptionExample {  
    public static void main(String[] args) {  
        try {  
  
            int[] arr = {1, 2, 3};  
            System.out.println(arr[3]);  
        } catch (ArrayIndexOutOfBoundsException e) {  
            System.out.println("Caught: " + e);  
        }  
    }  
}
```

Question 3: FinallyExample

```
class FinallyExample {  
    public static void main(String[] args) {  
        try {  
            int a = 10 / 0;
```

```
} catch (ArithmeticException e) {  
System.out.println("Caught Exception");  
} finally {  
System.out.println("Finally block always executes");  
}  
}  
}
```

Question 4: InvalidAgeException

```
class InvalidAgeException extends Exception {  
InvalidAgeException(String message) {  
super(message);  
}  
}
```

```
class AgeExceptionDemo {  
  
public static void main(String[] args) {  
int age = 15;  
try {  
if (age < 18)  
throw new InvalidAgeException("Age must be 18+");  
System.out.println("Valid Age");  
} catch (InvalidAgeException e) {  
System.out.println("Caught: " + e.getMessage());  
}  
}  
}
```

Question 5: MultipleCatchDemo

```
class MultipleCatchDemo {  
public static void main(String[] args) {
```

```
try {  
    int[] arr = new int[3];  
    arr[5] = 10 / 0;  
} catch (ArithmeticException e) {  
    System.out.println("Arithmetic Exception");  
} catch (ArrayIndexOutOfBoundsException e) {  
    System.out.println("Array Index Exception");  
}  
}  
}
```

Question 6: ThreadDemo

```
class MyThread extends Thread {  
    public void run() {  
  
        System.out.println("Thread is running...");  
    }  
}
```

```
class ThreadDemo {  
    public static void main(String[] args) {  
        MyThread t1 = new MyThread();  
        t1.start();  
    }  
}
```

Question 7: RunnableDemo

```
class MyRunnable implements Runnable {  
    public void run() {  
        System.out.println("Runnable Thread Running...");  
    }  
}
```



```
class RunnableDemo {  
    public static void main(String[] args) {  
        Thread t1 = new Thread(new MyRunnable());  
        t1.start();  
    }  
}
```

Question 8: ThreadInfo

```
class ThreadInfo {  
    public static void main(String[] args) {  
        Thread t = Thread.currentThread();  
  
        System.out.println("Name: " + t.getName());  
        System.out.println("Priority: " + t.getPriority());  
    }  
}
```

Question 9: ThrowsExample

```
class ThrowsExample {  
    static void check(int num) throws ArithmeticException {  
        if (num < 0)  
            throw new ArithmeticException("Negative number");  
    }  
  
    public static void main(String[] args) {  
        try {  
            check(-5);  
        } catch (ArithmeticException e) {  
            System.out.println(e.getMessage());  
        }  
    }  
}
```

```
}
```

Question 10: BankBalanceException

```
class LowBalanceException extends Exception {
```

```
    LowBalanceException(String msg) {
```

```
        super(msg);
```

```
    }
```

```
}
```

```
class BankBalanceException {
```

```
    public static void main(String[] args) {
```

```
        double balance = 300;
```

```
        try {
```

```
            if (balance < 500)
```

```
                throw new LowBalanceException("Balance too low!");
```

```
        } catch (LowBalanceException e) {
```

```
            System.out.println(e.getMessage());
```

```
        }
```

```
    }
```

```
}
```

Question 11: SyncDemo

```
class Counter {
```

```
    int count = 0;
```

```
    synchronized void increment() { count++; }
```

```
}
```

```
class SyncDemo extends Thread {
```

```
    static Counter c = new Counter();
```

```
    public void run() {
```

```
for (int i = 0; i < 1000; i++) c.increment();  
}
```

```
public static void main(String[] args) throws InterruptedException  
{  
    SyncDemo t1 = new SyncDemo();  
    SyncDemo t2 = new SyncDemo();  
  
    t1.start(); t2.start();  
    t1.join(); t2.join();  
    System.out.println("Count: " + c.count);  
}  
}
```

Question 12: DaemonThreadDemo

```
class DaemonThreadDemo extends Thread {  
    public void run() {  
        if (isDaemon())  
            System.out.println("Daemon thread running...");  
        else  
            System.out.println("User thread running...");  
    }  
}
```

```
public static void main(String[] args) {  
    DaemonThreadDemo t1 = new DaemonThreadDemo();  
    t1.setDaemon(true);  
    t1.start();  
}  
}
```

Question 13: JoinExample

```
class JoinExample extends Thread {
```

```

public void run() {
    for (int i = 1; i <= 3; i++) {
        System.out.println(Thread.currentThread().getName() + " -
" + i);
    }

}

```

```

public static void main(String[] args) throws InterruptedException
{
    JoinExample t1 = new JoinExample();
    JoinExample t2 = new JoinExample();
    t1.start();
    t1.join();
    t2.start();
}
}

```

Question 14: InterruptExample

```

class InterruptExample extends Thread {
    public void run() {
        try {
            Thread.sleep(1000);
            System.out.println("Completed");
        } catch (InterruptedException e) {
            System.out.println("Interrupted!");
        }
    }
}

```

```

public static void main(String[] args) {
    InterruptExample t = new InterruptExample();
}

```

```
t.start();  
t.interrupt();  
}
```

```
}
```

Question 15: RethrowExample

```
class RethrowExample {  
    static void validate() throws ArithmeticException {  
        try {  
            int x = 10 / 0;  
        } catch (ArithmeticException e) {  
            System.out.println("Caught inside method");  
            throw e;  
        }  
    }  
}
```

```
    public static void main(String[] args) {  
        try {  
            validate();  
        } catch (ArithmeticException e) {  
            System.out.println("Caught in main");  
        }  
    }  
}
```

Question 16: DeadlockExample

```
class DeadlockExample {  
    public static void main(String[] args) {  
        final String res1 = "A";  
        final String res2 = "B";
```

```
Thread t1 = new Thread() -> {  
  
    synchronized (res1) {  
        System.out.println("Thread1 locked res1");  
        try { Thread.sleep(100); } catch (Exception e) {}  
        synchronized (res2) {  
            System.out.println("Thread1 locked res2");  
        }  
    }  
});
```

```
Thread t2 = new Thread() -> {  
    synchronized (res2) {  
        System.out.println("Thread2 locked res2");  
        try { Thread.sleep(100); } catch (Exception e) {}  
        synchronized (res1) {  
            System.out.println("Thread2 locked res1");  
        }  
    }  
});
```

```
t1.start(); t2.start();  
  
}
```

Question 17: BankSyncDemo

```
class Bank {  
    private int balance = 1000;  
  
    synchronized void deposit(int amount) {
```

```
balance += amount;
System.out.println(Thread.currentThread().getName() + "
deposited. Balance: " + balance);
}
}
```

```
class BankSyncDemo extends Thread {
static Bank b = new Bank();
```

```
public void run() { b.deposit(500); }
```

```
public static void main(String[] args) {
new BankSyncDemo().start();
new BankSyncDemo().start();
}
}
```

18.

```
class DaemonThreadDemo extends Thread {
public void run() {
if (Thread.currentThread().isDaemon())
System.out.println("Daemon thread running");
else
System.out.println("User thread running");
}
}
```

```
public static void main(String[] args) {
```

```
DaemonThreadDemo t1 = new DaemonThreadDemo();
t1.setDaemon(true);
```

```
t1.start();
```

```
DaemonThreadDemo t2 = new DaemonThreadDemo();
```

```
t2.start();
```

```
}
```

```
}
```

19.

```
class ThreadJoinDemo extends Thread {
```

```
public void run() {
```

```
for (int i = 1; i <= 3; i++) {
```

```
System.out.println(Thread.currentThread().getName() + " -  
" + i);
```

```
try { Thread.sleep(500); } catch (Exception e) {}
```

```
}
```

```
}
```

```
public static void main(String[] args) throws InterruptedException
```

```
{
```

```
ThreadJoinDemo t1 = new ThreadJoinDemo();
```

```
ThreadJoinDemo t2 = new ThreadJoinDemo();
```

```
t1.start();
```

```
t1.join(); // Main thread waits for t1
```

```
t2.start();
```

```
}
```

```
}
```

20.

```
class InterruptDemo extends Thread {
```



```

public void run() {
    try {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Running: " + i);
            Thread.sleep(500);
        }
    } catch (InterruptedException e) {
        System.out.println("Thread interrupted!");
    }
}

```

```

public static void main(String[] args) {
    InterruptDemo t1 = new InterruptDemo();
    t1.start();
    t1.interrupt(); // Interrupt thread
}
}

```

21.

```

class NestedTryCatch {
    public static void main(String[] args) {
        try {
            int a = 10 / 2;

            try {
                int arr[] = new int[3];
                arr[5] = 50;
            } catch (ArrayIndexOutOfBoundsException e) {
                System.out.println("Inner catch: " + e);
            }
        }
    }
}

```

```
} catch (ArithmeticException e) {  
System.out.println("Outer catch: " + e);  
}  
}  
}
```

22.

```
class RethrowDemo {  
static void check() throws ArithmeticException {  
try {  
int a = 10 / 0;  
} catch (ArithmeticException e) {  
System.out.println("Caught inside check().  
Rethrowing...");  
throw e;  
}  
}
```

```
public static void main(String[] args) {  
try {  
check();  
  
} catch (ArithmeticException e) {  
System.out.println("Caught in main(): " + e);  
}  
}  
}
```

23.

// File: Parent.java in package pack1

```
package pack1;

public class Parent {
    protected void display() {
        System.out.println("Protected method accessed through
inheritance.");
    }
}
```

```
// File: Child.java in package pack2
package pack2;
```

```
import pack1.Parent;

public class Child extends Parent {
    public static void main(String[] args) {
        Child obj = new Child();
        obj.display();
    }
}
```

24.

```
import static java.lang.Math.*;

class StaticImportDemo {
    public static void main(String[] args) {
        System.out.println("Square root: " + sqrt(16));
        System.out.println("Power: " + pow(2, 3));
    }
}
```

```
}
```

25.

```
class SyncBlockDemo {  
    void printTable(int n) {  
        synchronized (this) {  
            for (int i = 1; i <= 5; i++) {  
                System.out.println(n * i);  
            }  
            try {  
                Thread.sleep(400);  
            } catch (Exception e) {  
            }  
        }  
    }  
}
```

```
public static void main(String[] args) {  
    SyncBlockDemo obj = new SyncBlockDemo();  
  
    Thread t1 = new Thread(() -> obj.printTable(5));  
    Thread t2 = new Thread(() -> obj.printTable(100));  
    t1.start();  
    t2.start();  
}
```

26.

```
class MultiTask1 extends Thread {  
    public void run() {  
        System.out.println("Task 1: Printing documents");  
    }  
}
```

```
}  
}
```

```
class MultiTask2 extends Thread {  
    public void run() {  
        System.out.println("Task 2: Sending emails");  
    }  
}
```

```
public class MultiTaskDemo {  
    public static void main(String[] args) {  
        new MultiTask1().start();  
        new MultiTask2().start();  
    }  
}
```

27.

```
class Message {  
    private String msg;
```

```
    public synchronized void write(String msg) {  
        this.msg = msg;  
        notify();  
    }
```

```
    public synchronized void read() {  
        try {  
            wait();  
            System.out.println("Message received: " + msg);
```

```
} catch (InterruptedException e) {  
}  
}  
}
```

```
class Writer extends Thread {  
    Message m;
```

```
    Writer(Message m) {  
        this.m = m;  
    }
```

```
    public void run() {  
        m.write("Hello Thread Communication!");  
    }  
}
```

```
class Reader extends Thread {  
    Message m;
```

```
    Reader(Message m) {  
        this.m = m;  
    }
```

```
    public void run() {  
        m.read();  
    }  
}
```

```
public class ThreadCommDemo {
```

```

public static void main(String[] args) {
    Message m = new Message();
    new Reader(m).start();
    new Writer(m).start();
}
}

```

28.

```

class DeadlockDemo {
    String s1 = "Java";
    String s2 = "Thread";

    Thread t1=new Thread()-
    >{synchronized(s1){try{Thread.sleep(100);}catch(
    Exception e)
    {
    }synchronized(s2)
    {
    System.out.println("Thread 1 executed");
    }}});

    Thread t2=new Thread()-
    >{synchronized(s2){try{Thread.sleep(100);}catch(
    Exception e)
    {
    }synchronized(s1)
    {
    System.out.println("Thread 2 executed");
    }
    }
}

```

```
});
```

```
public static void main(String[] args) {  
    DeadlockDemo obj = new DeadlockDemo();  
    obj.t1.start();  
    obj.t2.start();  
  
    }  
}
```

29.

```
class ThreadGroupDemo implements Runnable {  
    public void run() {  
        System.out.println(Thread.currentThread().getName() + " is  
        running");  
    }  
  
    public static void main(String[] args) {  
        ThreadGroup group = new ThreadGroup("MyGroup");  
  
        Thread t1 = new Thread(group, new ThreadGroupDemo(), "Thread-  
        1");  
  
        Thread t2 = new Thread(group, new ThreadGroupDemo(), "Thread-  
        2");  
  
        t1.start();  
        t2.start();  
    }  
}
```



```
System.out.println("Active threads in group: " +  
group.activeCount());  
}  
}
```

30.

```
class BankAccount {  
private int balance = 1000;  
  
synchronized void deposit(int amount) {  
balance += amount;  
System.out.println(Thread.currentThread().getName() + "  
deposited: " + amount + ", Balance: " + balance);  
}  
}
```

```
class DepositThread extends Thread {  
BankAccount account;  
  
DepositThread(BankAccount account) {  
this.account = account;  
}  
  
public void run() {  
account.deposit(500);  
}  
}
```

Labsheet-8

Program 1: StudentBean.java

// JavaBean example with id, name, grade

=====

```
public class StudentBean implements java.io.Serializable {
    private int id;
    private String name;
    private double grade;
    public StudentBean() { }
    public StudentBean(int id, String name, double grade) {
        this.id = id;
        this.name = name;
        this.grade = grade;
    }
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public double getGrade() { return grade; }
    public void setGrade(double grade) { this.grade = grade; }
    @Override
    public String toString() {

        return "StudentBean{id=" + id + ", name=" + name + ", grade=" + grade
        + "}";
    }
    public static void main(String[] args) {
        StudentBean s = new StudentBean(1, "Alice", 9.1);
        System.out.println(s);
    }
}
```

```
}
```

```
=====
```

Program 2: RectangleBean.java

// JavaBean example with length, width, area and perimeter methods

```
=====
```

```
public class RectangleBean {  
    private double length;  
    private double width;  
    public RectangleBean() { }  
    public RectangleBean(double length, double width) {  
        this.length = length;  
        this.width = width;  
    }  
    public double getLength() { return length; }  
    public void setLength(double length) { this.length = length; }  
    public double getWidth() { return width; }  
    public void setWidth(double width) { this.width = width; }  
    public double getArea() { return length * width; }  
    public double getPerimeter() { return 2 * (length + width); }  
    @Override  
    public String toString() {  
        return "RectangleBean{length=" + length + ", width=" + width + "}";  
    }  
    public static void main(String[] args) {  
        RectangleBean r = new RectangleBean(5, 3);  
        System.out.println(r);  
        System.out.println("Area: " + r.getArea());  
        System.out.println("Perimeter: " + r.getPerimeter());  
    }  
}
```

```
}  
}
```

```
=====
```

Program 3: EmployeeBean.java

// Serializable JavaBean for employee details

```
=====
```

```
import java.io.Serializable;  
  
public class EmployeeBean implements Serializable {  
    private int empId;  
    private String empName;  
    private double salary;  
  
    public EmployeeBean() { }  
    public EmployeeBean(int empId, String empName, double salary) {  
        this.empId = empId;  
        this.empName = empName;  
        this.salary = salary;  
    }  
  
    public int getEmpId() { return empId; }  
    public void setEmpId(int empId) { this.empId = empId; }  
    public String getEmpName() { return empName; }  
    public void setEmpName(String empName) { this.empName = empName; }  
    public double getSalary() { return salary; }  
    public void setSalary(double salary) { this.salary = salary; }  
  
    @Override  
    public String toString() {  
        return "EmployeeBean{empId=" + empId + ", empName=" + empName + ",  
            salary=" + salary + "}";  
    }  
}
```

```

public static void main(String[] args) {
    EmployeeBean e = new EmployeeBean(101, "John", 55000);
    System.out.println(e);
}
}

```

=====

Program 4: DateBean.java

// JavaBean with read-only property currentDate

=====

```

import java.time.LocalDate;

public class DateBean {
    public LocalDate getCurrentDate() {
        return LocalDate.now();
    }

    public static void main(String[] args) {
        DateBean d = new DateBean();
        System.out.println("Today's date: " + d.getCurrentDate());
    }
}

```

=====

Program 5: UserBean.java

// JavaBean with write-only password property

=====

```

public class UserBean {
    private String password;

    public void setPassword(String password) {
        this.password = password;
    }
}

```

```

}

boolean checkPassword(String attempt) {
return password != null && password.equals(attempt);
}

public static void main(String[] args) {
UserBean u = new UserBean();
u.setPassword("secret123");
System.out.println("Password set successfully!");
System.out.println("Check: " + u.checkPassword("secret123"));
}
}

```

=====

Program 6: AccountBean.java

// Bean with PropertyChangeListener to track balance changes

=====

```

import java.beans.PropertyChangeListener;
import java.beans.PropertyChangeSupport;

public class AccountBean {
private double balance;

private final PropertyChangeSupport pcs = new PropertyChangeSupport(this);

public double getBalance() { return balance; }

public void setBalance(double newBalance) {
double old = this.balance;
this.balance = newBalance;
pcs.firePropertyChange("balance", old, newBalance);
}

public void addPropertyChangeListener(PropertyChangeListener listener) {
pcs.addPropertyChangeListener(listener);
}
}

```

```

public static void main(String[] args) {

    AccountBean account = new AccountBean();
    account.addPropertyChangeListener(evt -> {
        if ("balance".equals(evt.getPropertyName())) {
            System.out.println("Balance changed from " + evt.getOldValue() +
                " to " + evt.getNewValue());
        }
    });
    account.setBalance(1000);
    account.setBalance(2000);
}
}

```

=====

Program 7: StudentBeanIntrospector.java

// Program using Introspector to list properties and methods

=====

```

import java.beans.*;

public class StudentBeanIntrospector {
    public static void main(String[] args) throws Exception {
        BeanInfo info = Introspector.getBeanInfo(StudentBean.class,
            Object.class);
        System.out.println("Properties:");
        for (PropertyDescriptor pd : info.getPropertyDescriptors()) {
            System.out.println(" - " + pd.getName());
        }
    }
}

```

=====

Program 8: CarBean.java

// JavaBean representing a car with make, model, and year

=====

```
public class CarBean {
    private String make;
    private String model;
    private int year;

    public CarBean() { }
    public CarBean(String make, String model, int year) {
        this.make = make;
        this.model = model;
        this.year = year;
    }
    public String getMake() { return make; }
    public void setMake(String make) { this.make = make; }
    public String getModel() { return model; }
    public void setModel(String model) { this.model = model; }
    public int getYear() { return year; }
    public void setYear(int year) { this.year = year; }
    public static void main(String[] args) {
        CarBean c = new CarBean("Toyota", "Corolla", 2020);
        System.out.println(c.getMake() + " " + c.getModel() + " " +
            c.getYear());
    }
}
```

=====

Program 9: CircleBean.java


```
// JavaBean with radius, read-only area and circumference
=====

public class CircleBean {
    private double radius;

    public CircleBean(double radius) { this.radius = radius; }
    public double getRadius() { return radius; }
    public void setRadius(double radius) { this.radius = radius; }
    public double getArea() { return Math.PI * radius * radius; }
    public double getCircumference() { return 2 * Math.PI * radius; }

    public static void main(String[] args) {
        CircleBean c = new CircleBean(3.0);
        System.out.println("Area: " + c.getArea());
        System.out.println("Circumference: " + c.getCircumference());
    }
}
```

```
=====

Program 10: InetInfo.java
// Display computer's IP address and host name using InetAddress
=====

import java.net.InetAddress;

public class InetInfo {
    public static void main(String[] args) {
        try {
            InetAddress localhost = InetAddress.getLocalHost();
            System.out.println("Host name: " + localhost.getHostName());
            System.out.println("Host address (IP): " +
                localhost.getHostAddress());
        } catch (Exception e) {
```

```
e.printStackTrace();  
}  
}  
}
```

Program 11:

```
import java.net.*;  
  
public class HelloServer {  
    public static void main(String[] args) {  
        int port = 5000;  
        try (ServerSocket ss = new ServerSocket(port)) {  
            System.out.println("Server started on port " + port);  
            Socket s = ss.accept();  
            System.out.println("Client connected: " + s.getInetAddress());  
            s.close();  
  
        } catch (Exception e) {  
            e.printStackTrace();  
        }  
    }  
}
```

Program 12:

```
import java.io.*;  
import java.net.*;  
  
public class HelloClient {  
    public static void main(String[] args) {  
        try (Socket s = new Socket("localhost", 5000);  
            PrintWriter out = new PrintWriter(s.getOutputStream(), true)) {  
            out.println("Hello Server");  
            System.out.println("Message sent.");  
        } catch (Exception e) {
```

```
e.printStackTrace();  
}  
}  
}
```

Program 13:

```
import java.io.*;  
import java.net.*;  
  
public class HelloServerReceive {  
    public static void main(String[] args) {  
        int port = 5000;  
  
        try (ServerSocket ss = new ServerSocket(port)) {  
            System.out.println("Server listening on port " + port);  
  
            try (Socket s = ss.accept();  
                BufferedReader in = new BufferedReader(new  
                    InputStreamReader(s.getInputStream())) {  
                System.out.println("Client: " + s.getInetAddress());  
  
                String msg = in.readLine();  
  
                System.out.println("Received: " + msg);  
            }  
  
        } catch (Exception e) {  
            e.printStackTrace();  
        }  
    }  
}
```

Program 14:

```
import java.io.*;  
import java.net.*;  
  
public class EchoServer {  
    public static void main(String[] args) {
```

```

int port = 5001;

try (ServerSocket ss = new ServerSocket(port)) {
    System.out.println("EchoServer started on port " + port);
    while (true) {
        Socket s = ss.accept();
        new Thread(() -> handleClient(s)).start();
    }
} catch (Exception e) {
    e.printStackTrace();
}

static void handleClient(Socket s) {
    try (BufferedReader in = new BufferedReader(new
        InputStreamReader(s.getInputStream()));
        PrintWriter out = new PrintWriter(s.getOutputStream(), true)) {
        String msg;
        while ((msg = in.readLine()) != null) {
            System.out.println("Received: " + msg);
            out.println(msg);
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
}

```

Program 15:

```

import java.io.*;
import java.net.*;

public class EchoClient {

```

```

public static void main(String[] args) {
    try (Socket s = new Socket("localhost", 5001);
        BufferedReader console = new BufferedReader(new
            InputStreamReader(System.in));
        PrintWriter out = new PrintWriter(s.getOutputStream(), true);
        BufferedReader in = new BufferedReader(new
            InputStreamReader(s.getInputStream())) {
        System.out.println("Enter message:");
        String msg;
        while ((msg = console.readLine()) != null && !msg.isEmpty()) {
            out.println(msg);
            System.out.println("Echo: " + in.readLine());
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
}

```

Program 16:

```

import java.net.*;

public class MessageSender {
    public static void main(String[] args) {
        try (DatagramSocket socket = new DatagramSocket()) {
            byte[] data = "Ping".getBytes();
            InetAddress addr = InetAddress.getByName("localhost");
            DatagramPacket packet = new DatagramPacket(data, data.length, addr,
                6000);

            socket.send(packet);

```

```

System.out.println("Sent UDP message: Ping");
} catch (Exception e) {
e.printStackTrace();
}
}
}

```

Program 17:

```

import java.net.*;

public class MessageReceiver {

public static void main(String[] args) {

try (DatagramSocket socket = new DatagramSocket(6000)) {

byte[] buf = new byte[1024];

DatagramPacket packet = new DatagramPacket(buf, buf.length);

System.out.println("Waiting for UDP message...");

socket.receive(packet);

String msg = new String(packet.getData(), 0, packet.getLength());

System.out.println("Received: " + msg);

} catch (Exception e) {

e.printStackTrace();

}

}

}

```

Program 18:

```

import java.io.*;

import java.net.*;

public class WebReader {

public static void main(String[] args) {

```

```

try {
    URL url = new URL("http://example.com");
    URLConnection conn = url.openConnection();
    try (BufferedReader in = new BufferedReader(new
        InputStreamReader(conn.getInputStream()))) {
        String line;
        while ((line = in.readLine()) != null)
            System.out.println(line);
    }
} catch (Exception e) {
    e.printStackTrace();
}
}
}

```

Program 19:

```

import java.io.*;
import java.net.*;

public class FileDownloader {
    public static void main(String[] args) {
        String fileUrl = "https://example.com";
        String output = "output.html";
        try {
            URL url = new URL(fileUrl);
            URLConnection conn = url.openConnection();
            try (InputStream in = conn.getInputStream();
                FileOutputStream out = new FileOutputStream(output)) {
                byte[] buffer = new byte[1024];
                int bytes;
                while ((bytes = in.read(buffer)) != -1)
                    out.write(buffer, 0, bytes);
            }
        }
    }
}

```

```

}
System.out.println("File downloaded: " + output);
} catch (Exception e) {
e.printStackTrace();
}

}

}

```

Program 20:

```

import java.io.*;
import java.net.*;

public class MultiClientServer {
public static void main(String[] args) {
int port = 7000;
try (ServerSocket server = new ServerSocket(port)) {
System.out.println("Server started on port " + port);
while (true) {
Socket client = server.accept();
new Thread(new ClientHandler(client)).start();
}
} catch (Exception e) {
e.printStackTrace();
}
}

}

class ClientHandler implements Runnable {
private Socket client;

ClientHandler(Socket client) {
this.client = client;

```



```

}

public void run() {
    String threadName = Thread.currentThread().getName();
    System.out.println "[" + threadName + "] Connected: " +
        client.getRemoteSocketAddress());
    try (BufferedReader in = new BufferedReader(new
        InputStreamReader(client.getInputStream()))) {
        String msg;
        while ((msg = in.readLine()) != null)

            System.out.println "[" + threadName + "] " + msg);
    } catch (Exception e) {
        e.printStackTrace();
    }
}
}
}

```

Program 21:

```

import java.io.*;
import java.net.*;

public class MultiClientTestClient {
    public static void main(String[] args) {
        try (Socket socket = new Socket("localhost", 7000);
            PrintWriter out = new PrintWriter(socket.getOutputStream(), true))
        {
            out.println("Hello from client " +
                Thread.currentThread().getName());
            System.out.println("Message sent to server.");
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

}

}

Labsheet-9

```
// ===== Program1_CreateFileCheckExists.java =====
```

```
import java.io.File;

public class Program1_CreateFileCheckExists {

    public static void main(String[] args) {

        try {

            File file = new File("example.txt");

            if (file.createNewFile()) {

                System.out.println("File created: " + file.getName());

            } else {

                System.out.println("File already exists.");

            }

        } catch (Exception e) {

            e.printStackTrace();

        }

    }

}
```

```
// ===== Program2_DisplayFileInfo.java =====
```

```
import java.io.File;

public class Program2_DisplayFileInfo {

    public static void main(String[] args) {

        File file = new File("example.txt");

        if (file.exists()) {

            System.out.println("Name: " + file.getName());

            System.out.println("Path: " + file.getAbsolutePath());

            System.out.println("Size: " + file.length() + " bytes");

        } else {

            System.out.println("File does not exist.");

        }

    }

}
```

```

}
}
// ===== Program3_ListFilesFolders.java =====
import java.io.File;

public class Program3_ListFilesFolders {
    public static void main(String[] args) {
        File dir = new File(".");
        File[] files = dir.listFiles();
        if (files != null) {
            for (File f : files) {
                System.out.println((f.isDirectory() ? "[DIR] " : "[FILE] ") +
                    f.getName());
            }
        }
    }
}

// ===== Program4_CreateFolder.java =====
import java.io.File;

public class Program4_CreateFolder {
    public static void main(String[] args) {
        File folder = new File("MyFolder");
        if (folder.mkdir()) {
            System.out.println("Folder created successfully.");
        } else {
            System.out.println("Failed to create folder.");
        }
    }
}

// ===== Program5_WriteDataRandomAccessFile.java =====
import java.io.RandomAccessFile;

```

```
public class Program5_WriteDataRandomAccessFile {  
    public static void main(String[] args) {  
        try (RandomAccessFile raf = new RandomAccessFile("data.txt", "rw")) {  
            raf.writeUTF("Hello, RandomAccessFile!");  
            System.out.println("Data written successfully.");  
        } catch (Exception e) {  
            e.printStackTrace();  
        }  
    }  
}
```

```
// ===== Program6_ReadBytesRandomAccessFile.java =====
```

```
import java.io.RandomAccessFile;  
  
public class Program6_ReadBytesRandomAccessFile {  
    public static void main(String[] args) {  
        try (RandomAccessFile raf = new RandomAccessFile("data.txt", "r")) {  
            byte[] bytes = new byte[10];  
            raf.read(bytes, 0, 10);  
            System.out.println("Read bytes: " + new String(bytes));  
        } catch (Exception e) {  
            e.printStackTrace();  
        }  
    }  
}
```

```
// ===== Program7_ModifyContentRandomAccessFile.java =====
```

```
import java.io.RandomAccessFile;  
  
public class Program7_ModifyContentRandomAccessFile {  
    public static void main(String[] args) {  
        try (RandomAccessFile raf = new RandomAccessFile("data.txt", "rw")) {  
            raf.seek(6);
```

```
raf.writeUTF("Modified");
System.out.println("Content modified successfully.");
} catch (Exception e) {
e.printStackTrace();
}
}
}
```

```
// ===== Program8_CopyFile.java =====
```

```
import java.io.*;
public class Program8_CopyFile {
public static void main(String[] args) {
try (FileInputStream fis = new FileInputStream("source.txt");
FileOutputStream fos = new FileOutputStream("destination.txt")) {
int b;
while ((b = fis.read()) != -1) {
fos.write(b);
}
System.out.println("File copied successfully.");
} catch (Exception e) {
e.printStackTrace();
}
}
}
```

```
// ===== Program9_ReadTextFileBufferedReader.java =====
```

```
import java.io.*;
public class Program9_ReadTextFileBufferedReader {
public static void main(String[] args) {
try (BufferedReader br = new BufferedReader(new FileReader("text.txt")))
{
```

```

String line;
while ((line = br.readLine()) != null) {
    System.out.println(line);
}
} catch (Exception e) {
    e.printStackTrace();
}
}
}

// ===== Program10_WriteTextFileFileWriter.java =====
import java.io.FileWriter;

public class Program10_WriteTextFileFileWriter {
    public static void main(String[] args) {
        try (FileWriter fw = new FileWriter("output.txt")) {
            fw.write("Writing to a text file using FileWriter.");
            System.out.println("Write successful.");
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

// ===== Program11_ByteVsCharStream.java =====
import java.io.*;

public class Program11_ByteVsCharStream {
    public static void main(String[] args) {
        try {
            FileOutputStream fos = new FileOutputStream("byteOutput.txt");
            fos.write("Byte Stream Example".getBytes());
            fos.close();

            FileWriter fw = new FileWriter("charOutput.txt");

```

```
fw.write("Character Stream Example");  
fw.close();  
System.out.println("Byte and Character streams demonstrated.");  
} catch (Exception e) {
```

```
e.printStackTrace();
```

```
}
```

```
}
```

```
}
```

```
// ===== Program12_CountCharsWords.java =====
```

```
import java.io.*;
```

```
public class Program12_CountCharsWords {
```

```
public static void main(String[] args) {
```

```
try (FileReader fr = new FileReader("text.txt")) {
```

```
int ch, charCount = 0, wordCount = 0;
```

```
boolean inWord = false;
```

```
while ((ch = fr.read()) != -1) {
```

```
charCount++;
```

```
if (Character.isWhitespace(ch)) {
```

```
if (inWord) wordCount++;
```

```
inWord = false;
```

```
} else {
```

```
inWord = true;
```

```
}
```

```
}
```

```
if (inWord) wordCount++;
```

```
System.out.println("Characters: " + charCount);
```

```
System.out.println("Words: " + wordCount);
```

```
} catch (Exception e) {
```

```
e.printStackTrace();
```



```

}
}
}
// ===== Program13_SerializeStudent.java =====
import java.io.*;
class Student implements Serializable {
    String name;
    int age;
    Student(String name, int age) {
        this.name = name;
        this.age = age;
    }
}
public class Program13_SerializeStudent {
    public static void main(String[] args) {
        try (ObjectOutputStream oos = new ObjectOutputStream(new
        FileOutputStream("student.ser"))) {
            Student s = new Student("Alice", 22);

            oos.writeObject(s);
            System.out.println("Student serialized successfully.");
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
// ===== Program14_DeserializeStudent.java =====
import java.io.*;
class Student implements Serializable {
    String name;

```

```

int age;
}

public class Program14_DeserializeStudent {
    public static void main(String[] args) {
        try (ObjectInputStream ois = new ObjectInputStream(new
            FileInputStream("student.ser"))) {
            Student s = (Student) ois.readObject();
            System.out.println("Name: " + s.name + ", Age: " + s.age);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

// ===== Program15_SerializableMultipleFields.java =====
import java.io.*;

class Employee implements Serializable {
    int id;
    String name;
    double salary;
    Employee(int id, String name, double salary) {
        this.id = id;
        this.name = name;
        this.salary = salary;
    }
}

public class Program15_SerializableMultipleFields {
    public static void main(String[] args) {
        try (ObjectOutputStream oos = new ObjectOutputStream(new
            FileOutputStream("employee.ser"))) {
            Employee emp = new Employee(101, "John", 50000.0);

```

```
oos.writeObject(emp);  
System.out.println("Employee object serialized.");  
} catch (Exception e) {  
e.printStackTrace();  
}  
}  
}
```

Labsheet-10

```
import java.util.*;

public class CollectionsLabsDetailed {

    static Scanner sc = new Scanner(System.in);

    public static void main(String[] args) {

        while (true) {

            System.out.println("\n===== Java Collections Labs Menu =====");

            for (int i = 1; i <= 30; i++)

                System.out.println(i + ". Question " + i);

            System.out.println("0. Exit");

            System.out.print("Enter your choice: ");

            int choice = sc.nextInt();

            sc.nextLine(); // consume newline

            switch (choice) {

                case 1:

                    q1();

                    break;

                case 2:

                    q2();

                    break;

                case 3:

                    q3();

                    break;

                case 4:

                    q4();

                    break;

                case 5:

                    q5();

                    break;
```

case 6:

q6();

break;

case 7:

q7();

break;

case 8:

q8();

break;

case 9:

q9();

break;

case 10:

q10();

break;

case 11:

q11();

break;

case 12:

q12();

break;

case 13:

q13();

break;

case 14:

q14();

break;

case 15:

q15();

break;

case 16:

q16();

break;

case 17:

q17();

break;

case 18:

q18();

break;

case 19:

q19();

break;

case 20:

q20();

break;

case 21:

q21();

break;

case 22:

q22();

break;

case 23:

q23();

break;

case 24:

q24();

break;

case 25:

q25();

```
break;
case 26:
q26();
break;
case 27:
q27();
break;
case 28:
q28();
break;
case 29:
q29();
break;
case 30:
q30();
break;
case 0:
System.out.println("Exiting...");
System.exit(0);
default:
System.out.println("Invalid choice!");
break;
}
}
}

// 1. Create ArrayList of integers and print
static void q1() {
System.out.println("Creating an ArrayList of integers...");
List<Integer> list = new ArrayList<>(Arrays.asList(1, 2, 3, 4, 5));
System.out.println("ArrayList elements: " + list);
```

```
}
```

```
// 2. Add elements and display size
```

```
static void q2() {  
    System.out.println("Adding elements to an ArrayList...");  
    List<Integer> list = new ArrayList<>();  
    list.add(10);  
    list.add(20);  
    list.add(30);  
    System.out.println("Elements added: " + list);  
    System.out.println("Current size of ArrayList: " + list.size());  
}
```

```
// 3. Insert element at specific index
```

```
static void q3() {  
    List<Integer> list = new ArrayList<>(Arrays.asList(1, 2, 3));  
    System.out.println("Original List: " + list);  
    System.out.println("Inserting 99 at index 1...");  
    list.add(1, 99);  
    System.out.println("Updated List: " + list);  
}
```

```
// 4. Remove element by index and value
```

```
static void q4() {  
    List<Integer> list = new ArrayList<>(Arrays.asList(1, 2, 3, 4));  
    System.out.println("Original List: " + list);  
    System.out.println("Removing element at index 2...");  
    list.remove(2);  
    System.out.println("Removing element with value 1...");  
    list.remove(Integer.valueOf(1));  
    System.out.println("Updated List: " + list);  
}
```



```

// 5. Update element at index
static void q5() {
    List<Integer> list = new ArrayList<>(Arrays.asList(1, 2, 3));
    System.out.println("Original List: " + list);
    System.out.println("Updating element at index 1 to 99...");
    list.set(1, 99);
    System.out.println("Updated List: " + list);
}

// 6. Iterate using for-each loop
static void q6() {
    List<String> list = Arrays.asList("A", "B", "C");
    System.out.println("Iterating using for-each loop:");
    for (String s : list)
        System.out.println("Element: " + s);
}

// 7. Iterate using Iterator
static void q7() {
    List<String> list = new ArrayList<>(Arrays.asList("X", "Y", "Z"));
    System.out.println("Iterating using Iterator:");
    Iterator<String> it = list.iterator();
    while (it.hasNext()) {
        String s = it.next();
        System.out.println("Element: " + s);
    }
}

// 8. Sort ArrayList of integers
static void q8() {
    List<Integer> list = new ArrayList<>(Arrays.asList(5, 1, 3, 2));
    System.out.println("Original List: " + list);
}

```

```

System.out.println("Sorting ArrayList in ascending order...");
Collections.sort(list);
System.out.println("Sorted List: " + list);
}

// 9. Reverse ArrayList of strings
static void q9() {
List<String> list = new ArrayList<>(Arrays.asList("One", "Two",
"Three"));
System.out.println("Original List: " + list);
System.out.println("Reversing the List...");
Collections.reverse(list);
System.out.println("Reversed List: " + list);
}

// 10. LinkedList of strings and print
static void q10() {
LinkedList<String> list = new LinkedList<>(Arrays.asList("A", "B",
"C"));
System.out.println("LinkedList elements: " + list);
}

// 11. Add elements at beginning and end of LinkedList
static void q11() {
LinkedList<String> list = new LinkedList<>(Arrays.asList("X", "Y"));
System.out.println("Original LinkedList: " + list);
System.out.println("Adding 'Start' at the beginning and 'End' at the
end...");
list.addFirst("Start");
list.addLast("End");
System.out.println("Updated LinkedList: " + list);
}

```

// 12. Remove first and last elements

```
static void q12() {  
    LinkedList<String> list = new LinkedList<>(Arrays.asList("A", "B",  
    "C"));  
    System.out.println("Original LinkedList: " + list);  
    System.out.println("Removing first and last elements...");  
    list.removeFirst();  
    list.removeLast();  
    System.out.println("Updated LinkedList: " + list);  
}
```

// 13. Retrieve first and last elements

```
static void q13() {  
    LinkedList<String> list = new LinkedList<>(Arrays.asList("A", "B",  
    "C"));  
    System.out.println("LinkedList: " + list);  
    System.out.println("First element: " + list.getFirst());  
    System.out.println("Last element: " + list.getLast());  
}
```

// 14. HashSet with duplicate elements

```
static void q14() {  
    System.out.println("Creating HashSet with duplicates: 1,2,2,3");  
    Set<Integer> set = new HashSet<>(Arrays.asList(1, 2, 2, 3));  
    System.out.println("HashSet elements (duplicates removed automatically):  
    " + set);  
}
```

// 15. Remove element from HashSet

```
static void q15() {  
    Set<Integer> set = new HashSet<>(Arrays.asList(1, 2, 3));  
    System.out.println("Original HashSet: " + set);  
    System.out.println("Removing element 2...");  
}
```

```

set.remove(2);

System.out.println("Updated HashSet: " + set);
}

// 16. Iterate HashSet using Iterator
static void q16() {
    Set<String> set = new HashSet<>(Arrays.asList("A", "B", "C"));
    System.out.println("Iterating HashSet using Iterator:");
    Iterator<String> it = set.iterator();
    while (it.hasNext())
        System.out.println("Element: " + it.next());
}

// 17. TreeSet with random elements
static void q17() {
    Set<Integer> set = new TreeSet<>(Arrays.asList(5, 2, 9, 1));
    System.out.println("TreeSet automatically sorts elements: " + set);
}

// 18. Check if TreeSet contains element
static void q18() {
    Set<Integer> set = new TreeSet<>(Arrays.asList(1, 2, 3));
    System.out.println("TreeSet elements: " + set);
    System.out.println("Does TreeSet contain 2? " + set.contains(2));
}

// 19. LinkedHashSet to maintain insertion order
static void q19() {
    Set<String> set = new LinkedHashSet<>(Arrays.asList("B", "A", "C"));
    System.out.println("LinkedHashSet maintains insertion order: " + set);
}

// 20. Compare HashSet, LinkedHashSet, TreeSet
static void q20() {

```

```

Set<Integer> hs = new HashSet<>(Arrays.asList(3, 1, 2));
Set<Integer> lhs = new LinkedHashSet<>(Arrays.asList(3, 1, 2));
Set<Integer> ts = new TreeSet<>(Arrays.asList(3, 1, 2));
System.out.println("HashSet (unordered): " + hs);
System.out.println("LinkedHashSet (insertion order): " + lhs);
System.out.println("TreeSet (sorted): " + ts);
}

// 21. PriorityQueue of integers
static void q21() {
    Queue<Integer> pq = new PriorityQueue<>();
    pq.add(5);
    pq.add(2);
    pq.add(8);
    System.out.println("PriorityQueue elements (natural order for removal):
" + pq);
}

// 22. Remove elements from PriorityQueue
static void q22() {
    Queue<Integer> pq = new PriorityQueue<>();
    pq.add(5);
    pq.add(2);
    pq.add(8);
    System.out.println("Original PriorityQueue: " + pq);

    System.out.println("Removing the head element...");
    pq.remove();
    System.out.println("After removal: " + pq);
}

// 23. Peek first element of PriorityQueue
static void q23() {

```

```
Queue<Integer> pq = new PriorityQueue<>();
pq.add(5);
pq.add(2);
pq.add(8);
System.out.println("PriorityQueue: " + pq);
System.out.println("Peek (head element without removal): " + pq.peek());
}
```

// 24. Queue reference using LinkedList

```
static void q24() {
Queue<String> q = new LinkedList<>();
System.out.println("Adding elements to Queue: A, B");
q.add("A");
q.add("B");
System.out.println("Queue elements: " + q);
System.out.println("Removing head element: " + q.remove());
System.out.println("Queue after removal: " + q);
}
```

// 25. Add, remove, peek in Queue

```
static void q25() {
Queue<String> q = new LinkedList<>();
System.out.println("Adding elements using offer(): X, Y");
q.offer("X");
q.offer("Y");
System.out.println("Queue: " + q);
System.out.println("Peek: " + q.peek());
System.out.println("Polling (removing head element): " + q.poll());
System.out.println("Queue after poll: " + q);
}
```

// 26. ArrayList of custom objects

```
static void q26() {
```

```

List<StudentC> list = new ArrayList<>();
list.add(new StudentC(1, "Alice"));
list.add(new StudentC(2, "Bob"));
System.out.println("Iterating custom objects using Iterator:");
Iterator<StudentC> it = list.iterator();
while (it.hasNext())
System.out.println(it.next());
}

```

```

// 27. Sort ArrayList of custom objects
static void q27() {
List<StudentC> list = new ArrayList<>();
list.add(new StudentC(2, "Bob"));
list.add(new StudentC(1, "Alice"));
System.out.println("Original List: " + list);
Collections.sort(list, (a, b) -> a.id - b.id);
System.out.println("Sorted by ID: " + list);
}

```

```

// 28. Convert List to Set
static void q28() {
List<Integer> list = Arrays.asList(1, 2, 2, 3);
System.out.println("Original List: " + list);
Set<Integer> set = new HashSet<>(list);
System.out.println("Converted to Set (duplicates removed): " + set);
}

```

```

// 29. Convert Set to List
static void q29() {
Set<String> set = new HashSet<>(Arrays.asList("A", "B", "C"));
System.out.println("Original Set: " + set);
List<String> list = new ArrayList<>(set);

```

```

System.out.println("Converted to List: " + list);
}
// 30. Clear all elements from List, Set, Queue
static void q30() {
List<Integer> list = new ArrayList<>(Arrays.asList(1, 2, 3));
Set<String> set = new HashSet<>(Arrays.asList("A", "B"));
Queue<String> queue = new LinkedList<>(Arrays.asList("X", "Y"));
System.out.println("Clearing all collections...");
list.clear();
set.clear();
queue.clear();
System.out.println("List after clear: " + list);
System.out.println("Set after clear: " + set);
System.out.println("Queue after clear: " + queue);
}
}
// Custom class for questions 26-27
class StudentC {
int id;
String name;
StudentC(int id, String name) {

this.id = id;
this.name = name;
}
public String toString() {
return "Student[id=" + id + ", name=" + name + "]";
}
}
}

```